AI-BASED TRIP PLANNER WEB APPLICATION

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

By

Oinam Dinibash Singh **(2201CS0105)**

Laba Moirangthem **(2201CS0104)**

Deemson Pukhrambam **(2201CS0118)**

Mathiuthailiu Panmei **(2301CS0302)**

Under the Guidance of

Mr. Golmei Shaheamlung
Assistant Professor, CSE, MTU

Department of Computer Science & Engineering

Manipur Technical University

June, 2026

DEDICATION

We dedicate this thesis to everyone who inspired and supported us throughout this journey.

To our parents and families: For your unwavering love, patience, and encouragement during the long hours of development and research. Your belief in us kept us motivated.

To our supervisor, Mr. Golmei Shaheamlung: For your invaluable guidance, technical mentorship, and constant support that shaped this project from concept to completion.

To our faculty and university: For providing the academic environment, resources, and opportunities that made this project possible.

To our friends and classmates: For the shared ideas, late-night debugging sessions, and constant motivation through every challenge.

To the open-source community: Whose libraries, frameworks, and tools – Django, Scikit-learn, and countless others – formed the foundation of our work.

This work stands as a testament to the collective effort, guidance, and support we received from all of you throughout our academic journey.

AI-BASED TRIP PLANNER WEB APPLICATION

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

By

Oinam Dinibash Singh **(2201CS0105)**

Laba Moirangthem **(2201CS0104)**

Deemson Pukhrambam **(2201CS0118)**

Mathiuthailiu Panmei **(2301CS0302)**

Under the Guidance of

Mr. Golmei Shaheamlung
Assistant Professor, CSE, MTU

Department of Computer Science & Engineering

Manipur Technical University

June, 2026

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

BONAFIDE CERTIFICATE

This is to certify that the thesis entitled **"AI-Based Trip Planner Web Application"** submitted by:

Oinam Dinibash Singh	**(2201CS0105)**
Laba Moirangthem	**(2201CS0104)**
Deemson Pukhrambam	**(2201CS0118)**
Mathiuthailiu Panmei	**(2301CS0302)**

to the **Department of Computer Science & Engineering**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree in **Computer Science and Engineering**, is a genuine and original work carried out by them under my supervision during the academic year **2024–2025**.

The project embodies the results of their own research and development work and has not been submitted, either in part or in full, for the award of any other degree or diploma of this university or any other institution. The work has been carried out with sincerity, academic integrity, and adherence to research ethics.

I consider this work worthy of consideration for the partial fulfilment of the requirements for the said degree.

Supervisor

Mr. Golmei Shaheamlung
Assistant Professor,
Department of Computer Science & Engineering
Manipur Technical University

Head of Department

Mrs. Aerena Tayenjam
HOD and Assistant Professor,
Department of Computer Science & Engineering
Manipur Technical University

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I declare that this project entitled **"AI-Based Trip Planner Web Application"**, submitted in partial fulfillment of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of original work carried out jointly by us under the supervision of **Mr. Golmei Shaheamlung**, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Date:
Place: Imphal

Oinam Dinibash Singh
Reg. No. – 2201CS0105
Department of Computer Science & Engineering
Manipur Technical University

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I declare that this project entitled **"AI-Based Trip Planner Web Application"**, submitted in partial fulfillment of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of original work carried out jointly by us under the supervision of **Mr. Golmei Shaheamlung**, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Date:
Place: Imphal

Laba Moirangthem
Reg. No. – 2201CS0104
Department of Computer Science
& Engineering
Manipur Technical University

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I declare that this project entitled **"AI-Based Trip Planner Web Application"**, submitted in partial fulfillment of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of original work carried out jointly by us under the supervision of **Mr. Golmei Shaheamlung**, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Date:
Place: Imphal

Deemson Pukhrambam
Reg. No. – 2201CS0118
Department of Computer Science
& Engineering
Manipur Technical University

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I declare that this project entitled **"AI-Based Trip Planner Web Application"**, submitted in partial fulfillment of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of original work carried out jointly by us under the supervision of **Mr. Golmei Shaheamlung**, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Date:
Place: Imphal

Mathiuthailiu Panmei
Reg. No. – 2301CS0302
Department of Computer Science & Engineering
Manipur Technical University

ACKNOWLEDGEMENT

We sincerely express our gratitude to everyone who supported us in the successful completion of our project entitled **"AI-Based Trip Planner Web Application."**

We are deeply thankful to our Project Supervisor, **Mr. Golmei Shaheamlung**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University, for his valuable guidance, continuous support, and encouragement throughout the project.

We also extend our sincere thanks to **W. Chandbahu Singh**, Vice Chancellor of Manipur Technical University, for providing a motivating academic environment and the necessary facilities.

We are grateful to **Mrs. Aerena Tayenjam**, Head of the Department and Project Coordinator, for her encouragement and support during the completion of this work.

We also thank all faculty members, friends, and our family members for their constant motivation and support. Above all, we thank the Almighty for giving us the strength and determination to complete this project successfully.

Sincerely,

Oinam Dinibash Singh
Laba Moirangthem
Deemson Pukhrambam
Mathiuthailiu Panmei

Date:
Place: Imphal

ABSTRACT

The advancement of artificial intelligence and machine learning has significantly transformed the way people plan and manage travel experiences. Traditional trip planning often involves extensive manual research related to destinations, budgeting, accommodations, climate conditions, transportation, and itinerary scheduling, making the process time-consuming and inefficient. To overcome these challenges, this project presents the development of an **AI-Based Trip Planner Web Application** that provides intelligent and personalized travel recommendations using machine learning techniques and modern web technologies.

The application is developed using the **Django** framework for backend development and utilizes **Scikit-learn** to implement the recommendation engine. The system accepts user inputs including travel budget, number of days, preferred climate type, and travel category, and predicts the most suitable Indian travel destination using a **Decision Tree Classifier** trained on a synthetic dataset of approximately 5,000 travel records covering 36 Indian destinations.

Following destination prediction, the system dynamically generates a structured day-wise itinerary including activity schedules, cost breakdowns, and travel guidance. The application incorporates **Django REST Framework** for API development, supports secure user authentication, and provides personalized dashboard features including trip history management with soft-delete functionality.

The frontend is built with **HTML5**, **CSS3**, and **JavaScript** following responsive design principles to support seamless access across desktop, tablet, and mobile devices. Testing and evaluation confirm that the recommendation engine achieves high classification accuracy with fast real-time inference, making the system suitable for practical deployment.

The project demonstrates the effective integration of artificial intelligence, machine learning, and full-stack web development to deliver a comprehensive, user-centric travel planning platform. Future enhancements include global dataset expansion, real-time price API integration, collaborative filtering, and progressive web application support.

Table of Contents

List of Tables

List of Figures

Symbols and Acronyms

$\hat{y}$	Predicted label by the classification model
J	Total number of classes in the classification task
p_i	Probability of an item belonging to class i
X	Feature matrix (input variables)
y	Target label vector (output variable)
AI	Artificial Intelligence
API	Application Programming Interface
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DBMS	Database Management System
DL	Deep Learning
DOM	Document Object Model
DT	Decision Tree
ERD	Entity-Relationship Diagram
GB	Gigabyte (unit of digital storage)

Gini Gini Impurity metric used in Decision Tree splitting

Git Distributed version control system

HTML HyperText Markup Language

HTTP Hypertext Transfer Protocol

HTTPS Hypertext Transfer Protocol Secure

IDE Integrated Development Environment

JS JavaScript

JSON JavaScript Object Notation

JWT JSON Web Token

KNN K-Nearest Neighbours

MB Megabyte (unit of digital storage)

ML Machine Learning

ms Millisecond (unit of time for inference latency)

MVC Model-View-Controller

MVT Model-View-Template (Django architecture pattern)

NLP Natural Language Processing

ORM Object-Relational Mapper

pip Package Installer for Python

PKL Pickle serialization file format (used for `.pkl` model files)

PWA Progressive Web Application

RAM Random Access Memory

REST Representational State Transfer

RF Random Forest

SPA Single Page Application

SQL Structured Query Language

SVM Support Vector Machine

UI User Interface

URL Uniform Resource Locator

UX User Experience

Chapter 1

Introduction

1.1 Background

The travel and tourism industry has undergone a major transformation over the last decade due to the rapid growth of digital technologies, internet accessibility, and artificial intelligence. Modern travelers increasingly depend on online platforms to discover destinations, compare travel costs, analyze weather conditions, book accommodations, and create travel itineraries. The emergence of smart technologies has shifted the tourism experience from agent-dependent planning to personalized, self-driven decision-making supported by intelligent systems [1].

Traditional tourism relied heavily on the expertise of travel agents who manually analyzed customer requirements such as destination preferences, travel duration, and budget limitations. These systems were limited by human capacity, static packages, and the absence of data-driven insights. The rise of the internet brought online booking platforms such as TripAdvisor and MakeMyTrip, which improved accessibility but still placed the burden of itinerary planning on the user [2].

The increasing complexity of travel decisions and the vast amount of available information have created a need for intelligent systems that can automatically process user constraints and generate personalized travel recommendations. Machine learning provides the computational tools necessary to build such systems. Algorithms such as Decision Trees, Random Forests, and collaborative filtering techniques have shown strong performance in preference-based classification tasks relevant to travel planning [3].

This project presents the design, development, and evaluation of an **AI-Based Trip Planner Web Application** that addresses these gaps by integrating machine learning-based recommendation, dynamic itinerary generation, and full-stack web

development within a unified platform.

1.2 Problem Statement

Despite the growth of digital travel platforms, the following challenges persist:

- Users must manually search through large volumes of destination data before making decisions, leading to information overload and decision fatigue.
- Existing systems lack personalized, constraint-aware recommendations that simultaneously account for budget, climate preference, trip duration, and travel category.
- No widely available platform provides end-to-end trip planning from destination recommendation to day-wise itinerary generation within a single interface.
- Many tourism recommendation systems are limited by cold-start problems or require extensive historical user data to function effectively.

1.3 Objectives

The primary objectives of this project are:

1. To develop an AI-powered destination recommendation system using a Decision Tree classifier trained on a curated synthetic dataset of 36 Indian travel destinations.
2. To implement a dynamic itinerary generation engine that produces day-wise travel schedules, cost breakdowns, and activity suggestions based on predicted destinations.
3. To build a complete, responsive, and secure full-stack web application using Django and Django REST Framework.
4. To provide personalized user features including trip dashboard, history management, and soft-delete functionality.
5. To evaluate the system's recommendation accuracy, inference speed, and scalability.

1.4 Scope of the Project

The AI-Based Trip Planner Web Application covers the following scope:

- Destination recommendation for 36 Indian travel destinations spanning all major states and union territories.
- User inputs: travel budget (INR), number of days, climate preference (Hot, Cold, Moderate), and travel type (Adventure, Relaxation, Cultural, Family).
- Automated day-wise itinerary generation with cost estimation and activity suggestions.
- Secure user authentication, registration, and session management.
- Personalized trip dashboard with history tracking, soft-delete, and trip restoration features.
- RESTful API for frontend-backend communication.
- Responsive web interface supporting desktop, tablet, and mobile devices.

1.5 Organization of the Thesis

The remainder of this thesis is organized as follows:

- **Chapter 2** presents a comprehensive review of related literature on travel planning systems, recommender systems, and machine learning applications in tourism.
- **Chapter 3** describes the system analysis and design, including the architecture, data flow, ERD, and technology stack.
- **Chapter 4** details the machine learning implementation covering dataset generation, feature engineering, the Decision Tree classifier, and model serialization.
- **Chapter 5** explains the web application development process using Django, covering authentication, itinerary generation, and REST API development.
- **Chapter 6** discusses the user interface and experience design, key application interfaces, and responsive design strategies.

- **Chapter 7** covers testing and evaluation including unit testing, integration testing, and model performance analysis.
- **Chapter 8** presents the conclusion and outlines future enhancement directions.

Chapter 2

Literature Review

2.1 Traditional Travel Planning

The history of travel planning dates back to the era when tourism services were exclusively provided through physical travel agencies. Traditional travel planning was a manual, agent-driven process in which customers visited travel agencies and described their preferences. Travel agents manually analyzed customer requirements such as destination preferences, travel duration, budget limitations, and seasonal conditions before recommending suitable travel packages. These services were often personalized because human agents could directly communicate with customers and understand their expectations in detail.

However, traditional travel planning methods suffered from several limitations. The process was time-consuming, expensive, and highly dependent on the expertise and experience of travel agents. Since recommendations were manually generated, they were susceptible to human error, personal bias, and limited information availability. Travel agencies also relied on static tour packages that lacked flexibility and customization. Customers often had to adjust their preferences according to pre-designed itineraries rather than receiving plans specifically tailored to their needs.

The introduction of the internet and web technologies transformed the tourism industry dramatically. During the era of Web 1.0 and later Web 2.0, online travel platforms emerged as alternatives to physical travel agencies. Websites such as TripAdvisor, MakeMyTrip, and other online booking systems digitized travel-related services including hotel reservations, transportation booking, destination reviews, and tourism information [1]. These platforms increased accessibility and convenience by allowing users to search for travel options directly from their devices.

Although digital travel platforms simplified booking procedures, they introduced new challenges. Users were required to manually browse through extensive lists of destinations, accommodations, and reviews before making decisions. The abundance of information often caused decision fatigue and information overload. While these systems provided search and filtering functionalities, they still relied heavily on user effort for itinerary planning and destination selection.

Furthermore, traditional digital tourism platforms generally focused on transactional services such as booking flights and hotels rather than providing intelligent planning assistance. They lacked the capability to understand complex user constraints and generate personalized travel recommendations automatically. As tourism demand continued to increase, researchers and developers recognized the need for intelligent systems capable of automating travel planning processes and reducing user workload through artificial intelligence and recommendation technologies [2].

2.2 Recommender Systems in Tourism

Recommender systems have become one of the most influential applications of artificial intelligence in modern digital platforms. These systems are widely used in e-commerce, entertainment, social media, and online marketplaces to analyze user preferences and provide personalized suggestions. Companies such as Amazon and Netflix have successfully utilized recommendation systems to improve customer engagement, user satisfaction, and service efficiency [2].

In the tourism industry, recommender systems play an important role in helping travelers discover destinations, accommodations, restaurants, and travel activities. The growing complexity of tourism-related information has made intelligent recommendation technologies essential for simplifying user decision-making processes.

2.2.1 Content-Based Filtering

Content-Based Filtering recommends items based on similarities between user preferences and item attributes. In tourism applications, this method analyzes destination characteristics such as climate, travel type, activities, cost range, and cultural features. If a user previously showed interest in beach destinations or adventure tourism, the system recommends destinations with similar attributes.

The main advantage of Content-Based Filtering is its ability to generate personalized recommendations without relying on other users' data. However, the

method often suffers from limited diversity because recommendations are restricted to items similar to those previously selected by the user. It may also struggle to recommend entirely new or unexplored destinations.

2.2.2 Collaborative Filtering

Collaborative Filtering generates recommendations based on similarities between users. The system identifies travelers with comparable interests and suggests destinations preferred by similar users. This technique is widely used because it can identify hidden patterns and relationships within large datasets [2].

Despite its popularity, Collaborative Filtering faces several limitations in tourism systems. It often suffers from the "cold start" problem, where recommendations become inaccurate when insufficient user data is available. In addition, tourism decisions are influenced by dynamic contextual factors such as weather, budget, travel duration, and seasonal conditions, which are difficult to capture using collaborative approaches alone.

2.2.3 Hybrid Recommendation Approaches

To overcome the limitations of individual recommendation methods, modern tourism systems increasingly adopt hybrid recommendation models that combine multiple techniques. Hybrid systems integrate content-based analysis, collaborative filtering, contextual information, and machine learning algorithms to improve recommendation accuracy and personalization [2].

In travel planning, users often specify both hard constraints and soft preferences. Hard constraints include fixed limitations such as budget, available travel days, and climate requirements. Soft preferences refer to subjective interests such as relaxation, adventure, cultural exploration, or family-friendly experiences. Traditional recommendation systems are not always effective at simultaneously processing these complex parameters.

As a result, supervised machine learning models and intelligent classification algorithms are gaining popularity in tourism recommendation research. These models treat destination recommendation as a predictive classification problem, enabling systems to generate more accurate and constraint-aware travel suggestions.

2.3 Machine Learning Applications in Travel

Machine learning has emerged as one of the most powerful technologies in modern intelligent systems, enabling applications to automatically learn patterns from data and improve decision-making capabilities without explicit programming. In the travel and tourism industry, machine learning techniques are increasingly used to enhance customer experiences, optimize travel recommendations, predict user behavior, and automate itinerary planning processes [3].

2.3.1 Decision Tree Algorithms

Decision Trees are supervised learning models that classify data based on decision rules derived from input features. In travel recommendation systems, Decision Trees can effectively process categorical and numerical inputs such as budget ranges, travel days, climate preferences, and trip types. They generate interpretable classification rules that make them particularly suitable for transparent recommendation systems [3].

Decision Trees use metrics such as **Gini Impurity** or **Information Gain** to determine the best feature splits at each node. The resulting tree structure provides clear decision paths, making it easy to trace and explain individual recommendations to users.

2.3.2 Random Forest and Ensemble Methods

Random Forest is an ensemble learning method that constructs multiple Decision Trees and aggregates their predictions for improved accuracy and robustness. Ensemble approaches generally outperform single classifiers by reducing variance and bias. However, the interpretability of individual tree predictions is reduced when multiple trees are combined.

2.3.3 Collaborative and Hybrid Systems

Research by Ricci, Rokach, and Shapira demonstrates that hybrid systems combining collaborative filtering and machine learning classifiers achieve superior performance in travel recommendation tasks compared to single-method approaches [2]. Such systems benefit from both the pattern recognition capabilities of machine learning and the personalization strengths of collaborative filtering.

2.4 Related Works and Research Gaps

Several travel recommendation platforms and academic studies have explored the use of machine learning for tourism:

- Buhalis and Law analyzed the transformation of eTourism research over two decades, noting the shift from static information systems toward dynamic, intelligent recommendation platforms [1].
- Ricci, Rokach, and Shapira provide a comprehensive handbook on recommender systems including tourism-specific applications, covering content-based, collaborative, and hybrid approaches [2].
- Django Software Foundation provides robust, production-ready framework documentation supporting scalable web application development for AI-integrated systems [4].

While existing systems demonstrate promising results in individual recommendation tasks, most lack the integration of end-to-end trip planning capabilities within a single platform. Specifically:

- No widely available system combines real-time destination recommendation, dynamic itinerary generation, and personalized dashboard management in a unified web application.
- Many systems require large historical user datasets, making them unsuitable for new platforms with limited user data.
- Most existing solutions do not address Indian travel destinations specifically with localized budget estimations and cultural travel categories.

This project addresses these gaps by developing a complete, constraint-aware AI-Based Trip Planner Web Application tailored for Indian travel destinations using a synthetic dataset and Decision Tree classification.

Chapter 3

System Analysis and Design

3.1 System Architecture

The AI-Based Trip Planner Web Application is designed using a modular, layered architecture that separates user interaction, business logic, machine learning inference, and data persistence into distinct components. This separation ensures maintainability, scalability, and testability of each system layer.

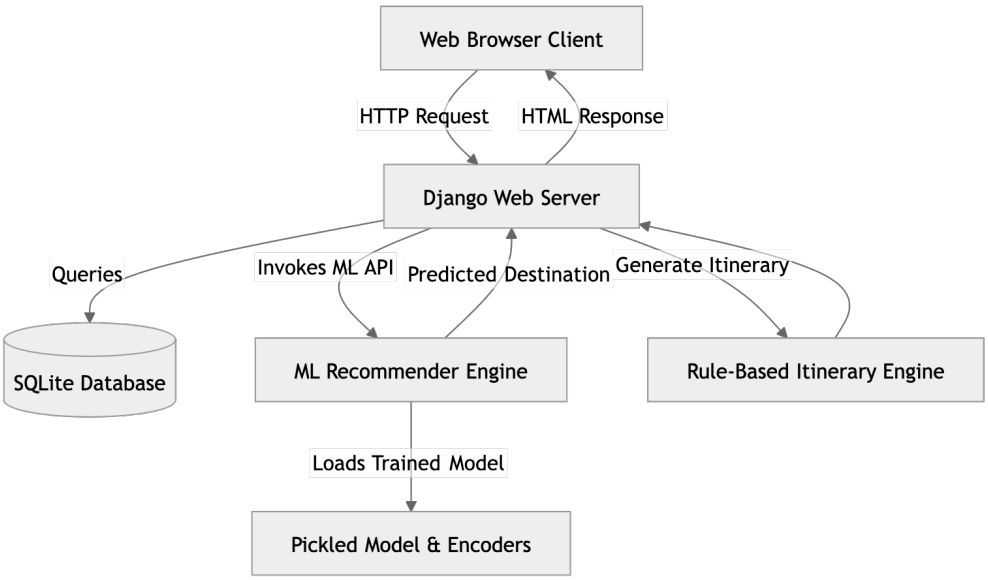


Figure 3.1: High-Level System Architecture

The architecture consists of the following layers:

- **Presentation Layer**: HTML5, CSS3, and JavaScript templates rendered through Django's template engine, providing responsive and interactive user interfaces.

- **Application Layer**: Django views and URL routing handle HTTP request processing, authentication, session management, and template rendering.
- **API Layer**: Django REST Framework exposes RESTful endpoints for frontend-backend communication, enabling JSON-based request and response handling.
- **Machine Learning Layer**: Pre-trained Decision Tree model loaded via Joblib for real-time destination prediction and itinerary generation.
- **Data Layer**: SQLite3 database accessed through Django's ORM for storing user accounts, trip records, and related data.

3.2 User Interaction Flow

The system follows a structured user interaction sequence from input submission to itinerary delivery:

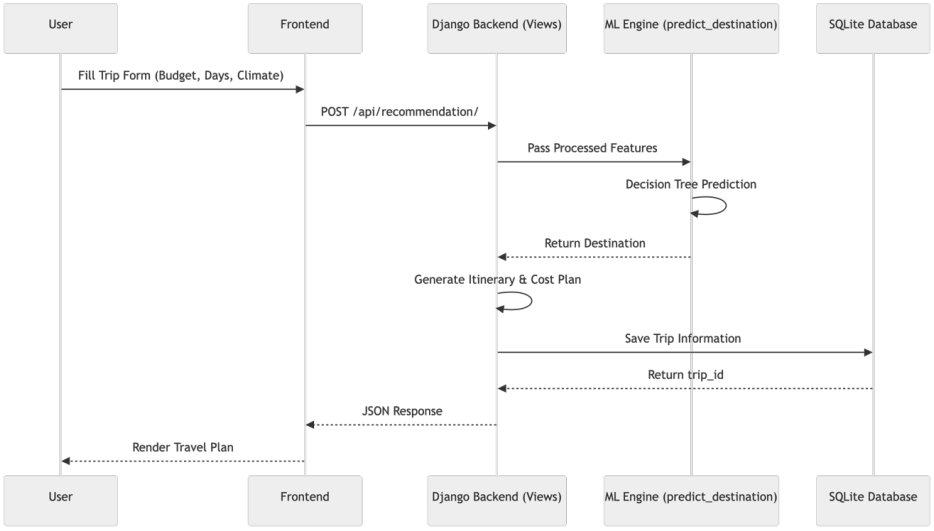
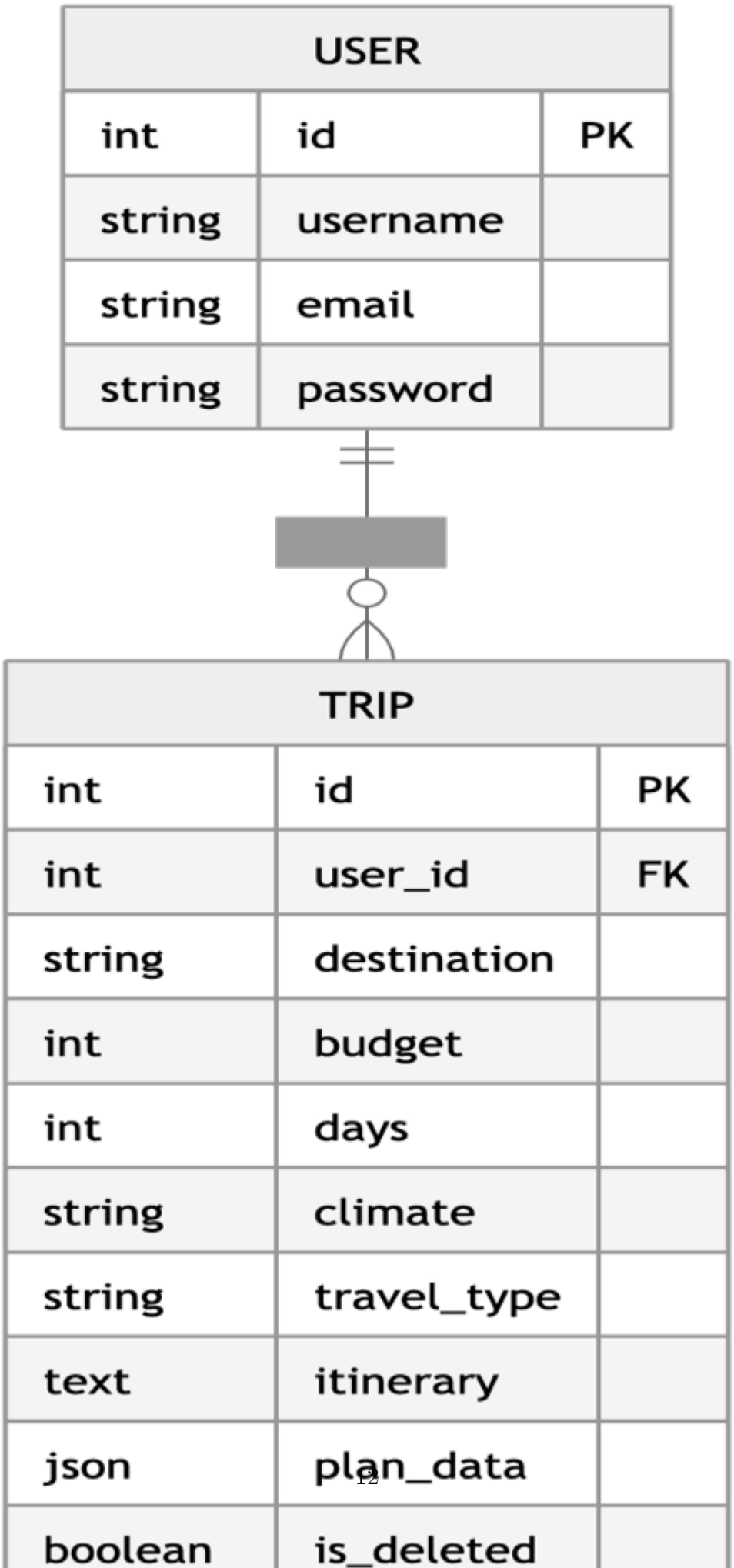


Figure 3.2: User Interaction Sequence Diagram

3.3 Database Design

3.3.1 Entity Relationship Diagram

The database design consists of two primary entities: `USER` and `TRIP`. The relationship is one-to-many, where one user can have multiple associated trip records.


USER
int
id
PK
string
username
string
email
string
password
TRIP
int
id
PK
int
user_id
FK
string
destination
int
budget
int
days
string
climate
string
travel_type
text
itinerary
json
plan_data
boolean
is_deleted

3.3.2 USER Entity

The `USER` table stores authentication and account-related information. Attributes include:

- **id**: Unique primary key
- **username**: User login identifier
- **email**: Registered email address
- **password**: Encrypted user password (managed by Django's `AbstractUser`)

3.3.3 TRIP Entity

The `TRIP` table stores all generated travel plans and itinerary information. Attributes include:

- **destination**: Predicted destination name
- **budget**: User-specified travel budget (INR)
- **days**: Trip duration in days
- **climate**: Preferred climate type (Hot, Cold, Moderate)
- **travel_type**: Selected travel category (Adventure, Cultural, Family, Relaxation)
- **itinerary**: Generated day-wise schedule (text)
- **plan_data**: JSON-formatted structured trip data including cost breakdown
- **is_deleted**: Soft-delete status flag
- **deleted_at**: Timestamp for soft-deletion event
- **created_at**: Timestamp of trip creation

3.4 Technology Stack

The AI-Based Trip Planner Web Application utilizes a combination of backend frameworks, machine learning libraries, frontend technologies, and database systems to provide a scalable and intelligent travel planning platform.

Table 3.1: Technology Stack Breakdown

Component	**Technology**	**Purpose**
Backend	Python, Django	Web framework, request routing, and ORM handling
Machine Learning	Scikit-Learn, Pandas	Data preprocessing and ML prediction model
Database	SQLite3	Persistent relational data storage
Frontend	HTML5, CSS3, JavaScript	User interface rendering and interaction
API Framework	Django REST Framework	JSON REST APIs and request serialization
Model Serialization	Joblib	Saving and reloading trained ML model files
Version Control	Git	Source code versioning and management

3.4.1 Backend Technologies

The backend is implemented using **Python** and **Django** [4]. Django provides secure authentication, database abstraction through ORM, URL routing, middleware support, and template rendering functionalities. Key advantages include:

- Rapid development through built-in utilities
- Built-in security (CSRF protection, password hashing)
- Scalable and modular architecture
- Easy database integration through ORM

3.4.2 Machine Learning Technologies

The machine learning module uses **Scikit-learn** [3], **Pandas** [5], and **NumPy** for:

- Data preprocessing and cleaning
- Categorical label encoding
- Decision Tree classifier training
- Real-time prediction inference

Chapter 4

Machine Learning Implementation

4.1 Dataset Generation and Preprocessing

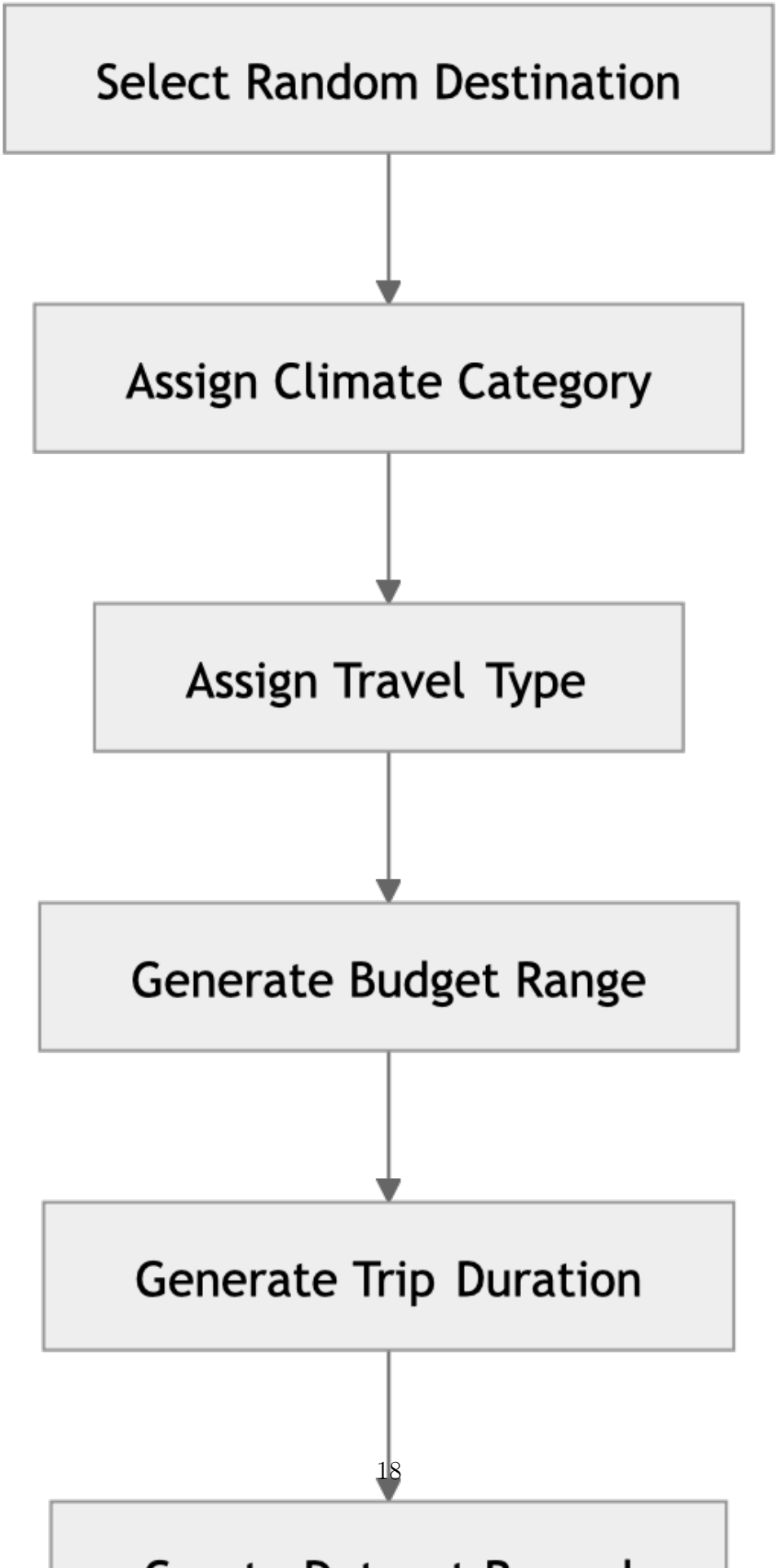
One of the major challenges in travel recommendation research is the lack of publicly available datasets containing structured mappings between travel constraints and personalized destinations. Since real-world tourism datasets are often incomplete, inconsistent, or commercially restricted, this project utilizes a carefully designed synthetic dataset generation approach [3].

The system includes a curated list of **36 Indian travel destinations** categorized according to:

- Climate type (Hot, Cold, Moderate)
- Travel category (Adventure, Cultural, Family, Relaxation, Religious)
- Budget range (minimum and maximum estimated cost in INR)
- Tourist attractiveness and suitability

To simulate realistic user behavior, the application generates approximately **5,000 synthetic travel records** using a randomized generation workflow. For each record, a destination is randomly selected from the curated list and a budget value is randomly sampled from within that destination's defined range. Trip duration is randomly assigned between 2 and 7 days.

4.1.1 Synthetic Dataset Generation Workflow



Each generated record contains the following features:

- **Budget**: Estimated travel cost in INR (sampled within destination-specific range)
- **Days**: Trip duration (integer, 2–7 days)
- **Climate**: Preferred climate type (categorical: Hot, Cold, Moderate)
- **Travel Type**: Travel category (categorical: Adventure, Cultural, Family, Relaxation)
- **Destination**: Target prediction label (one of 36 destination names)

4.2 Feature Engineering and Encoding

Machine learning algorithms require numerical input values. Therefore, categorical features such as climate type and travel category must be converted into numeric representations before model training. The project uses **LabelEncoder** from Scikit-learn to encode categorical variables [3].

Table 4.1: Feature Encoding Mapping

Feature	**Original Values**	**Encoded Values**
Climate	Hot, Cold, Moderate	0, 1, 2
Travel Type	Adventure, Cultural, Family, Relaxation	0, 1, 2, 3
Destination	36 destination names	0 to 35

The encoded features form the input matrix $X = [\text{budget}, \text{days}, \text{climate}, \text{travel\_type}]$ used for model training, while the encoded destination label forms the target vector y.

4.3 Decision Tree Classifier Algorithm

The system utilizes the **DecisionTreeClassifier** algorithm from Scikit-learn [3] with a maximum depth of 8. Decision Trees are non-parametric supervised learning algorithms that recursively split datasets into branches based on feature conditions. They are particularly suitable for travel recommendation systems because they can effectively handle both categorical and numerical variables while producing interpretable decision rules.

4.3.1 Advantages of Decision Trees

- Easy to interpret and explain to end users
- Fast inference speed for real-time predictions
- Handles categorical features efficiently after encoding
- Requires minimal preprocessing compared to deep learning models
- Suitable for multi-class classification with many target classes

4.3.2 Mathematical Basis: Gini Impurity

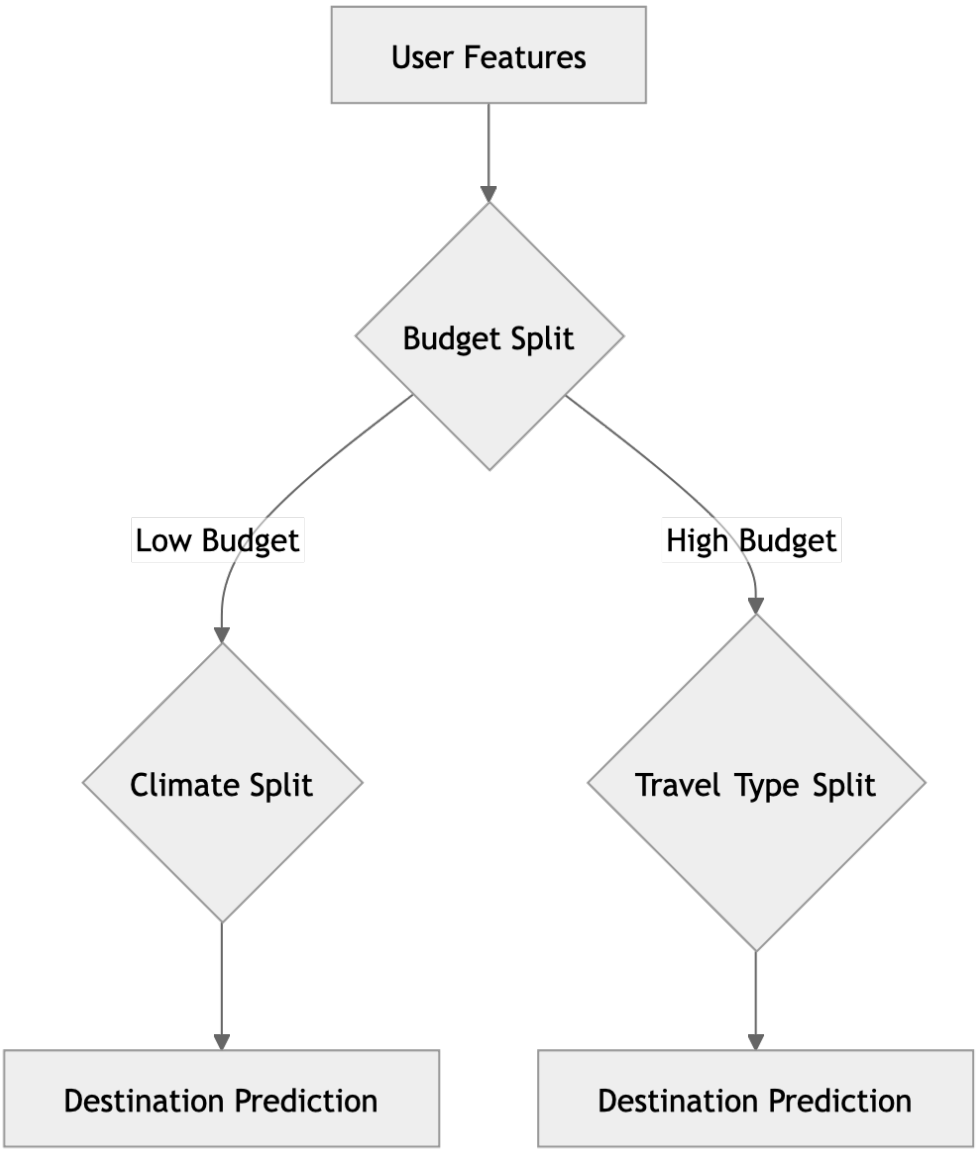


Figure 4.2: Decision Tree Splitting Logic Representation

The Decision Tree uses the **Gini Impurity** metric to determine the optimal feature split during training:

$$\text{Gini}(p) = 1 - \sum_{i=1}^{J} p_i^2 \tag{4.1}$$

Where:

- p_i represents the probability of an item belonging to class i
- J represents the total number of classes

Lower Gini impurity values indicate better node purity and more accurate classification splits. The algorithm recursively selects the feature and threshold that minimizes the weighted Gini impurity of the resulting child nodes.

4.4 Model Training and Serialization

After preprocessing and feature encoding, the dataset is divided into training and testing subsets. The model is trained using the `.fit(X, y)` method provided by Scikit-Learn:

```
from sklearn.tree import DecisionTreeClassifier
import joblib

model = DecisionTreeClassifier(
    max_depth=8,
    random_state=42
)
model.fit(X, y)

# Serialize the trained model
joblib.dump(model, "trip_model.pkl")
joblib.dump(le_climate, "climate_encoder.pkl")
joblib.dump(le_travel, "travel_encoder.pkl")
joblib.dump(le_destination, "destination_encoder.pkl")
```

Listing 4.1: Decision Tree Training Code

Figure 4.3: Training Pipeline

4.4.1 Model Serialization

Once training is completed, the trained model and encoders are serialized using the **Joblib** library. Serialization enables the application to reuse trained models without retraining during runtime. Serialized files include:

- `trip_model.pkl` – Trained Decision Tree classifier
- `climate_encoder.pkl` – LabelEncoder for climate feature
- `travel_encoder.pkl` – LabelEncoder for travel type feature
- `destination_encoder.pkl` – LabelEncoder for destination labels

These files are loaded by the Django backend during application startup, enabling fast real-time predictions and efficient deployment. The benefits of serialization include:

- Faster application startup (no re-training required)
- Reduced computational overhead at inference time
- Real-time recommendation support with sub-millisecond prediction latency
- Easier deployment and portability across environments
- Clear separation of training and inference environments

Chapter 5

Web Application Development

5.1 Django Framework and MVT Architecture

The AI-Based Trip Planner Web Application is developed using the **Django** framework [4], one of the most widely used Python-based web development frameworks. Django follows the **Model-View-Template (MVT)** architectural pattern, which separates the application into independent layers for data handling, business logic, and presentation rendering.

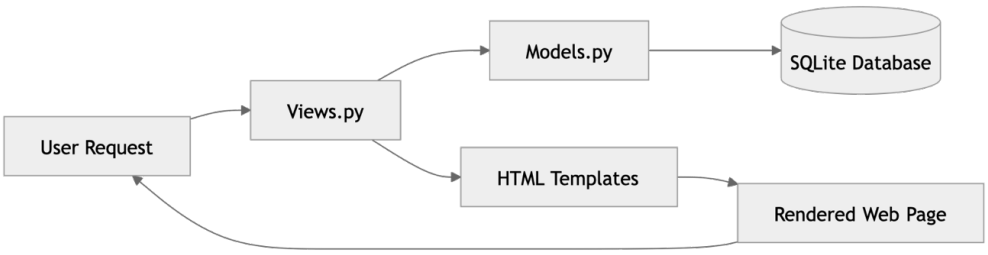


Figure 5.1: Django MVT Architecture

The MVT architecture improves code organization, scalability, and maintainability by enforcing separation of concerns:

- **Model**: Defines the database schema and handles data persistence through ORM queries.
- **View**: Contains the application business logic, processes HTTP requests, and returns responses.
- **Template**: Renders the HTML presentation layer with dynamic data injected by views.

5.1.1 Models Layer

The `Trip` model defines the database schema for storing trip records:

```
from django.db import models
from django.utils import timezone

class Trip(models.Model):
    CLIMATE_CHOICES = [
        ('hot', 'Hot'),
        ('cold', 'Cold'),
        ('moderate', 'Moderate'),
    ]
    TRAVEL_TYPE_CHOICES = [
        ('adventure', 'Adventure'),
        ('relaxation', 'Relaxation'),
        ('cultural', 'Cultural'),
        ('family', 'Family'),
    ]
    destination = models.CharField(max_length=255, default='Unknown')
    budget = models.IntegerField(default=0)
    days = models.IntegerField(default=1)
    climate = models.CharField(max_length=20, choices=CLIMATE_CHOICES
    )
    travel_type = models.CharField(max_length=20, choices=
    TRAVEL_TYPE_CHOICES)
    itinerary = models.TextField(blank=True)
    plan_data = models.JSONField(default=dict, blank=True)
    image = models.ImageField(upload_to='trip_images/', blank=True,
    null=True)
    is_deleted = models.BooleanField(default=False)
    deleted_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

Listing 5.1: Trip Model Definition

5.1.2 Views Layer

The views layer handles business logic and request processing. The **request handling workflow** operates as follows:

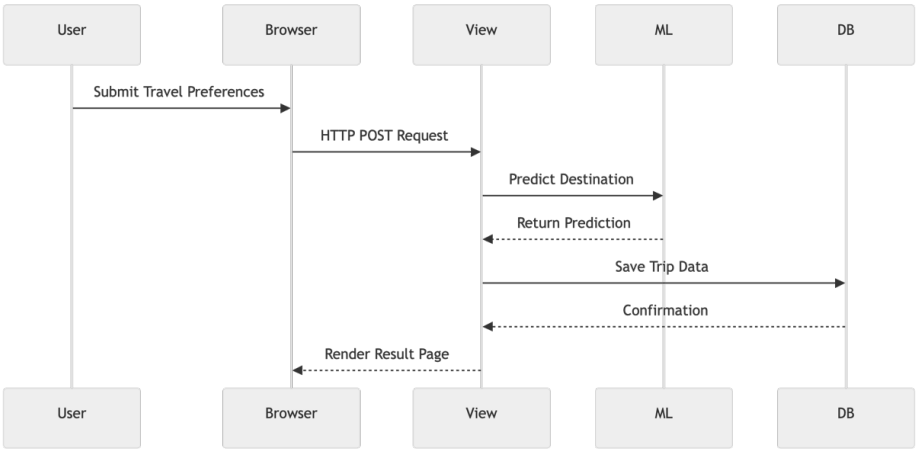


Figure 5.2: Request Handling Workflow

5.1.3 Templates Layer

Django templates render dynamic HTML pages using template tags and variables. The system uses a base template (`base.html`) with `{% extends %}` inheritance for consistent layout across all pages.

5.2 Authentication System

The application implements Django's built-in authentication framework with custom views for secure user registration, login, and session management.

The authentication workflow is as follows:

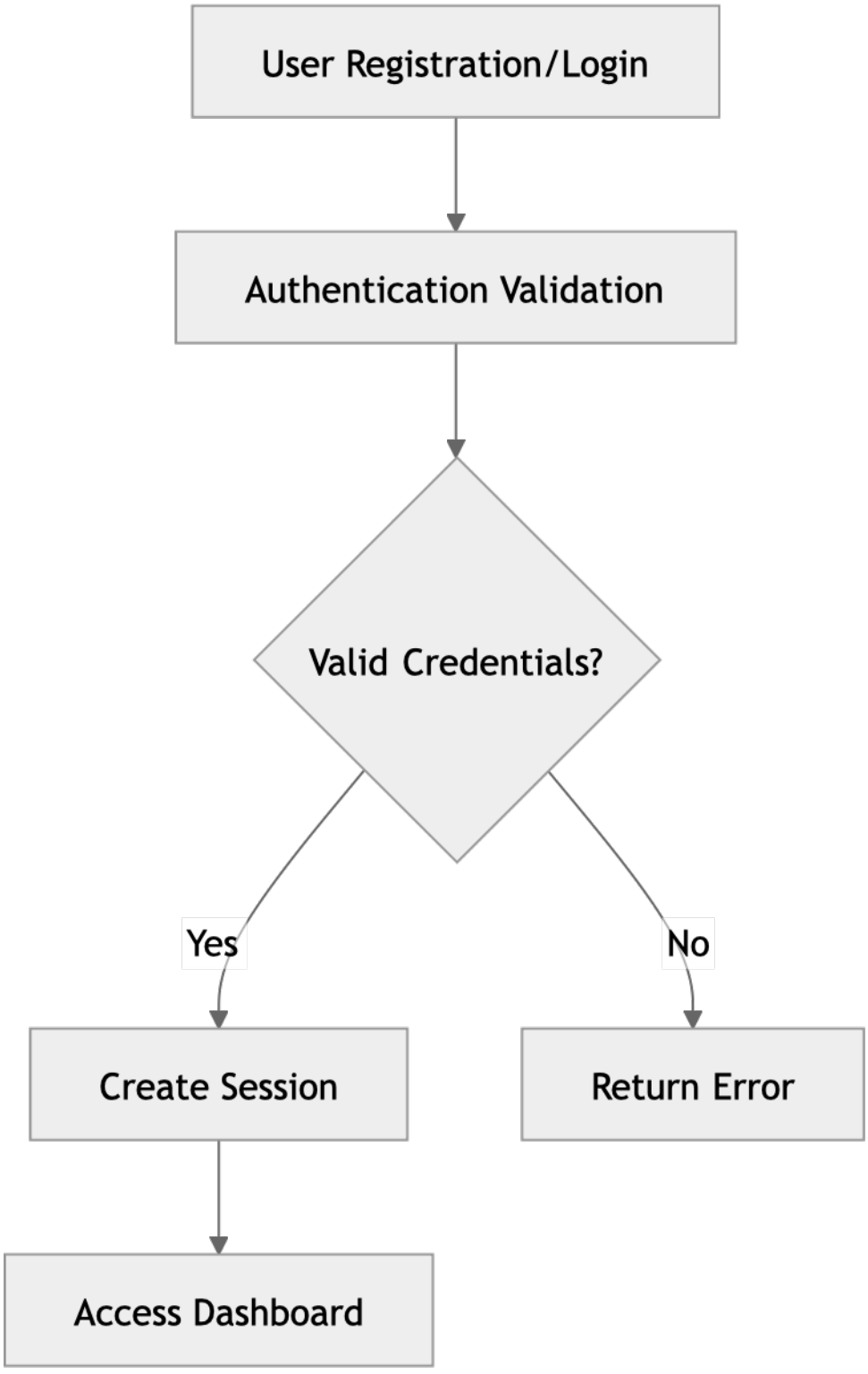


Figure 5.3: Authentication Workflow

5.2.1 User Registration

The registration view accepts a username, email, and password. Passwords are validated for strength and hashed using Django's default PBKDF2 algorithm before storage. After successful registration, users are automatically redirected to the login page.

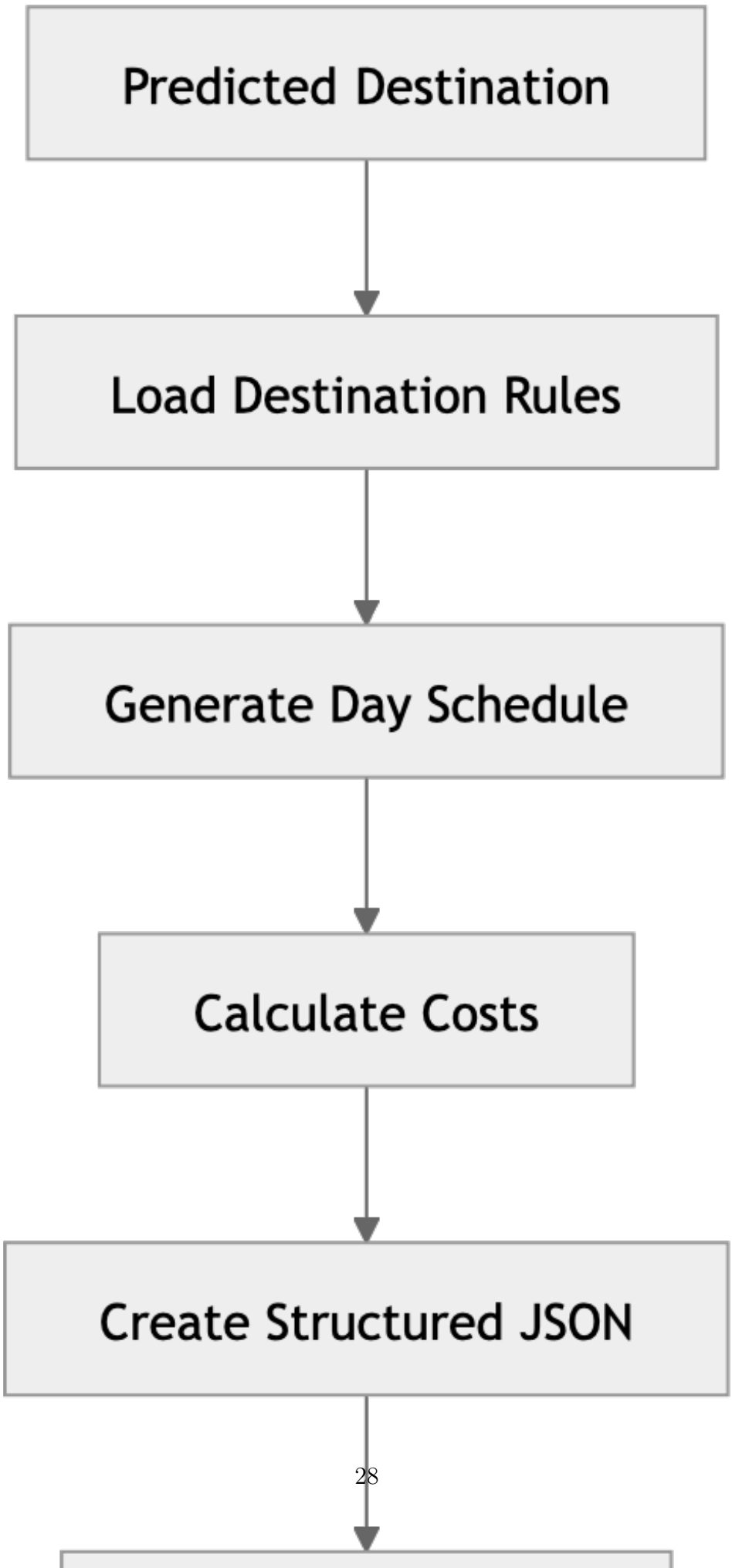
5.3 Itinerary Generation Engine

The itinerary generation system produces dynamic day-wise travel schedules based on the predicted destination and user-specified trip duration. Each destination in the system has a predefined activity database containing:

- Daily sightseeing activities
- Recommended dining and local experiences
- Suggested timings for each activity
- Local travel recommendations

The itinerary engine automatically:

- Truncates schedules for shorter trips (fewer days than the full itinerary)
- Extends schedules for longer trips by repeating activity cycles
- Rearranges activities dynamically to match trip duration


Predicted Destination
Load Destination Rules
Generate Day Schedule
Calculate Costs
Create Structured JSON

5.3.1 Cost Estimation Logic

The cost estimation module distributes the user's total budget across standard travel expense categories using fixed percentage allocations:

- **Accommodation**: 40%
- **Food**: 25%
- **Transportation**: 20%
- **Activities**: 15%

This breakdown totals exactly 100%, ensuring accurate financial planning in the generated itinerary.

5.4 API Integration and Django REST Framework

The application uses **Django REST Framework** for API development and JSON serialization. REST APIs enable communication between frontend interfaces and backend services while supporting scalable application design.

Figure 5.5: API Communication Architecture

5.4.1 Request Validation

The application uses `TripRequestSerializer` to validate incoming request data. Validation checks include:

- Required fields presence (budget, days, climate, travel_type)
- Numeric constraints (budget > 0, days between 1 and 30)
- Allowed climate values (hot, cold, moderate)
- Travel category validation (adventure, relaxation, cultural, family)

5.4.2 JSON Response Structure

The API returns structured JSON responses containing the predicted destination, budget breakdown, estimated expenses, daily itinerary, and travel metadata. An example response:

```
{
    "destination": "Manali",
    "budget": 25000,
    "days": 5,
    "itinerary": {
        "Day 1": "Arrival and local sightseeing",
        "Day 2": "Adventure sports and trekking",
        "Day 3": "Cultural exploration and shopping"
    },
    "cost_breakdown": {
        "accommodation": 10000,
        "food": 6250,
        "transportation": 5000,
        "activities": 3750
    }
}
```

Listing 5.2: API JSON Response Example

Table 5.1: API Endpoint Specifications

Endpoint	**Method**	**Payload**	**Response**
`/api/recommendation/`	POST	{budget, days, climate, travel_type}	JSON destination, itinerary, and cost breakdown
`/`	GET	—	Home page template
`/plan-trip/`	GET	—	Trip planning form template
`/results/`	GET	data query param	Results view template
`/my-trips/`	GET	—	Dashboard template
`/trips/<id>/delete/`	POST	CSRF token	Redirect to dashboard

Chapter 6

User Interface and Experience

6.1 Frontend Architecture

The frontend architecture focuses on responsive design, smooth navigation, and visually attractive layouts. Although the application uses traditional server-side rendering through Django templates, the interface is designed to simulate modern single-page application behavior through JavaScript-driven interactions.

Frontend technologies include:

- **HTML5**: Semantic markup and form structure
- **CSS3**: Custom styling, responsive grid layouts, and animations
- **JavaScript**: Asynchronous form submission via Fetch API and dynamic DOM manipulation
- **Django Template Language**: Server-side data binding and template inheritance

Figure 6.1: Frontend Rendering Workflow

6.1.1 Responsive Design Features

The application supports a wide range of devices:

- **Desktop**: Full-width layouts with multi-column grids

- **Tablets**: Adaptive column collapsing with fluid widths
- **Smartphones**: Single-column stacked layouts with touch-friendly navigation

Responsive layouts are achieved through:

- Flexible grid systems using CSS Flexbox and Grid
- Adaptive image scaling with `max-width: 100%`
- Media queries for breakpoint-specific styling
- Mobile-friendly navigation with hamburger menu support

6.1.2 Dynamic Destination Images

To improve visual appeal, the application integrates fallback destination images using the **Unsplash Source API**. If local images are unavailable, the system dynamically loads travel-related images based on destination names. This feature enhances visual engagement, user immersion, and destination attractiveness.

6.2 Key Application Interfaces

6.2.1 Landing and Authentication Pages

The landing page acts as the entry point of the application and focuses on user engagement, travel inspiration, easy navigation, and quick registration access. The page features a hero section with a full-width background, a destination carousel showcasing popular Indian travel locations, and an inline trip planning form.

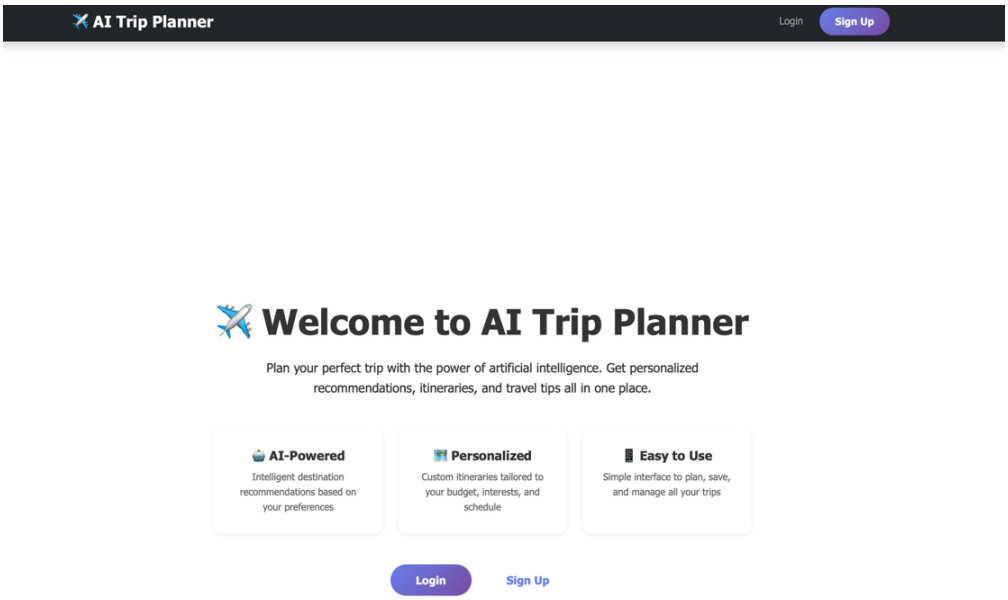


Figure 6.2: Home Page Interface

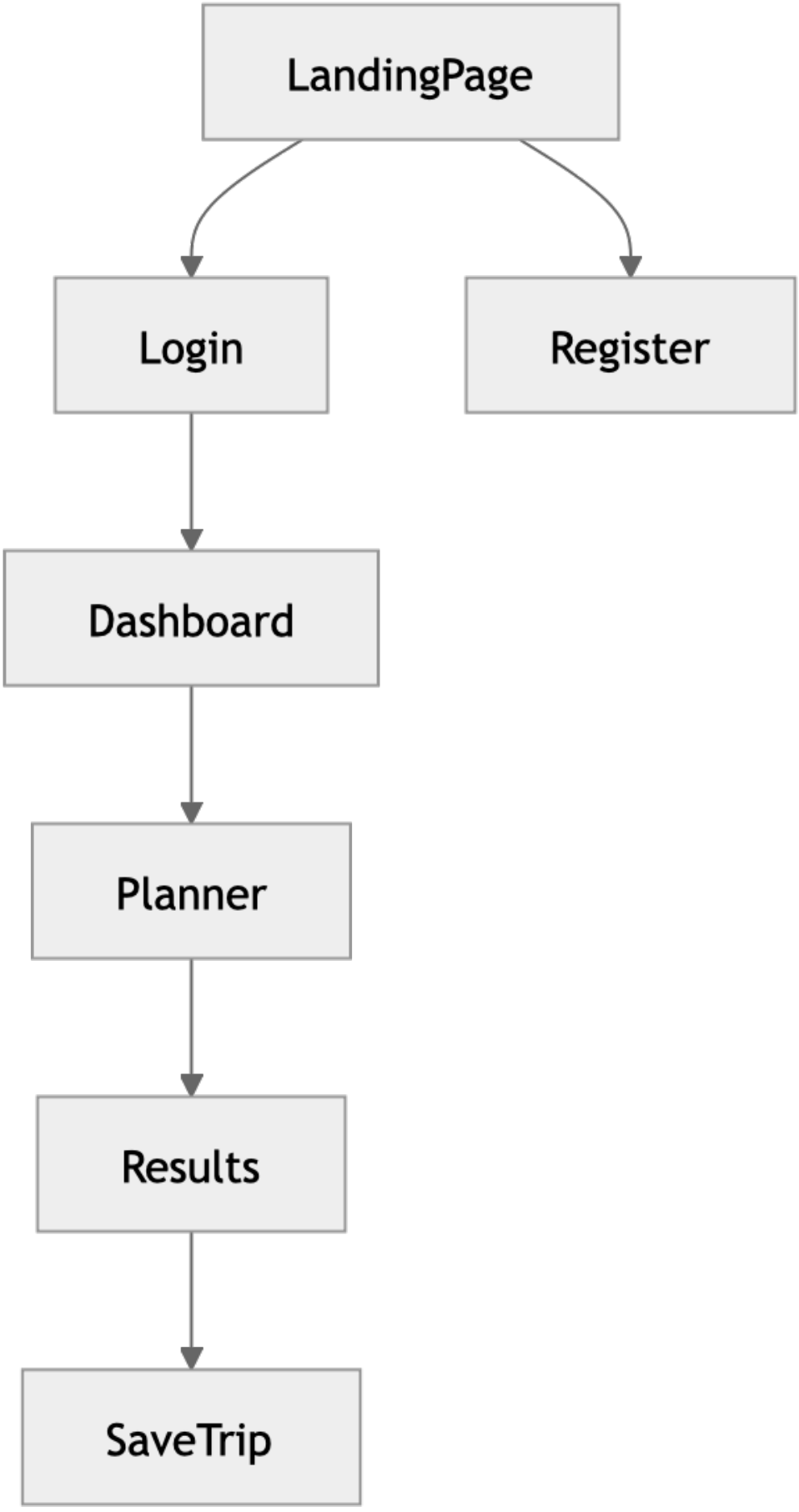


Figure 6.3: User Navigation Structure

Authentication pages include:

- **Login form**: Username and password fields with CSRF protection
- **Registration form**: Username, email, and password with confirmation
- **Password validation**: Minimum length and confirmation matching
- **Session management**: Persistent login sessions with secure cookies

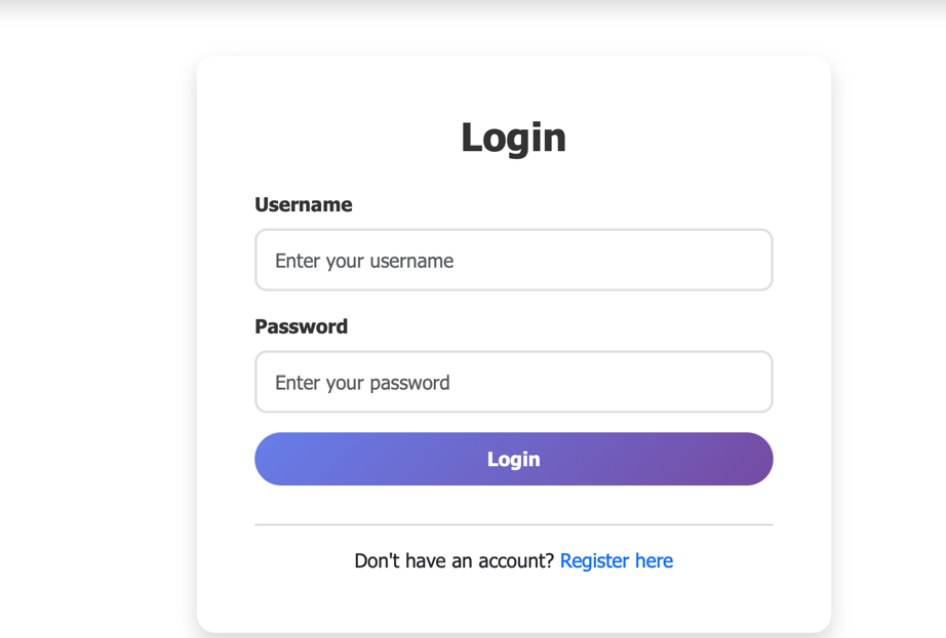


Figure 6.4: Login Interface

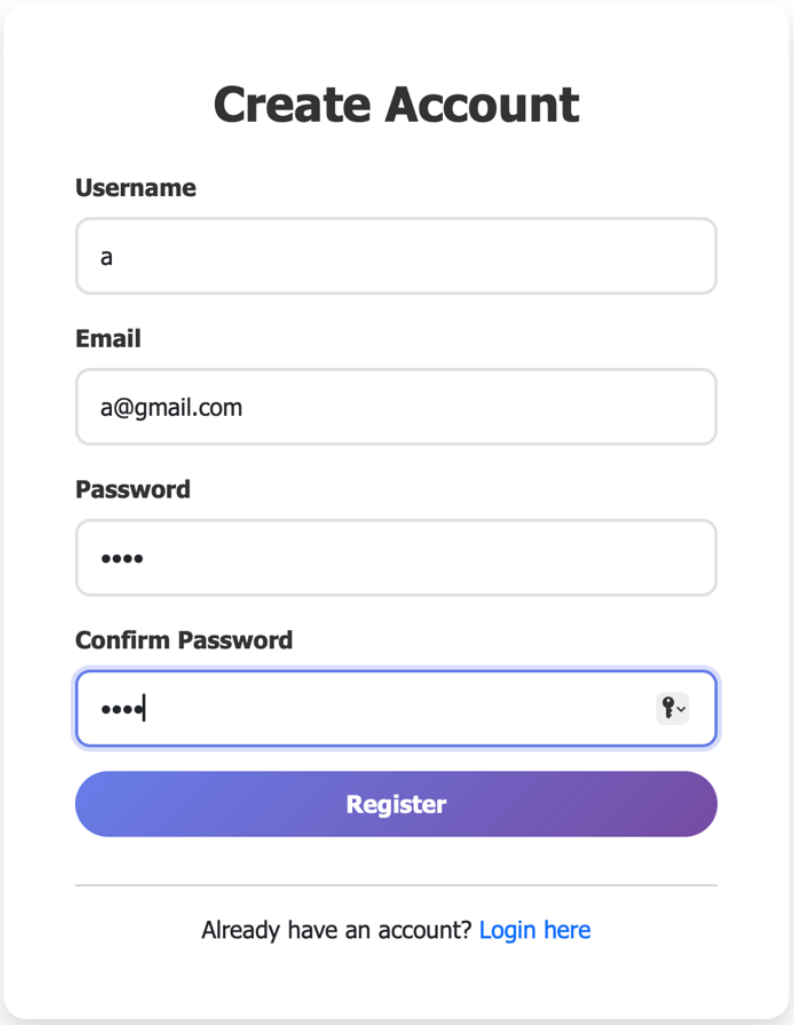


Figure 6.5: Registration Interface

6.2.2 Destinations Explorer

The destination explorer displays all 36 travel destinations using responsive card-based grid layouts. Each destination card contains:

- Destination image (local or dynamically loaded from Unsplash)
- Climate information badge
- Estimated budget range
- Travel category tag
- Short destination description
- Quick link to the trip planning form pre-filled with the destination

The explorer supports real-time client-side filtering by destination name search and climate type selection, enabling users to find relevant destinations efficiently.

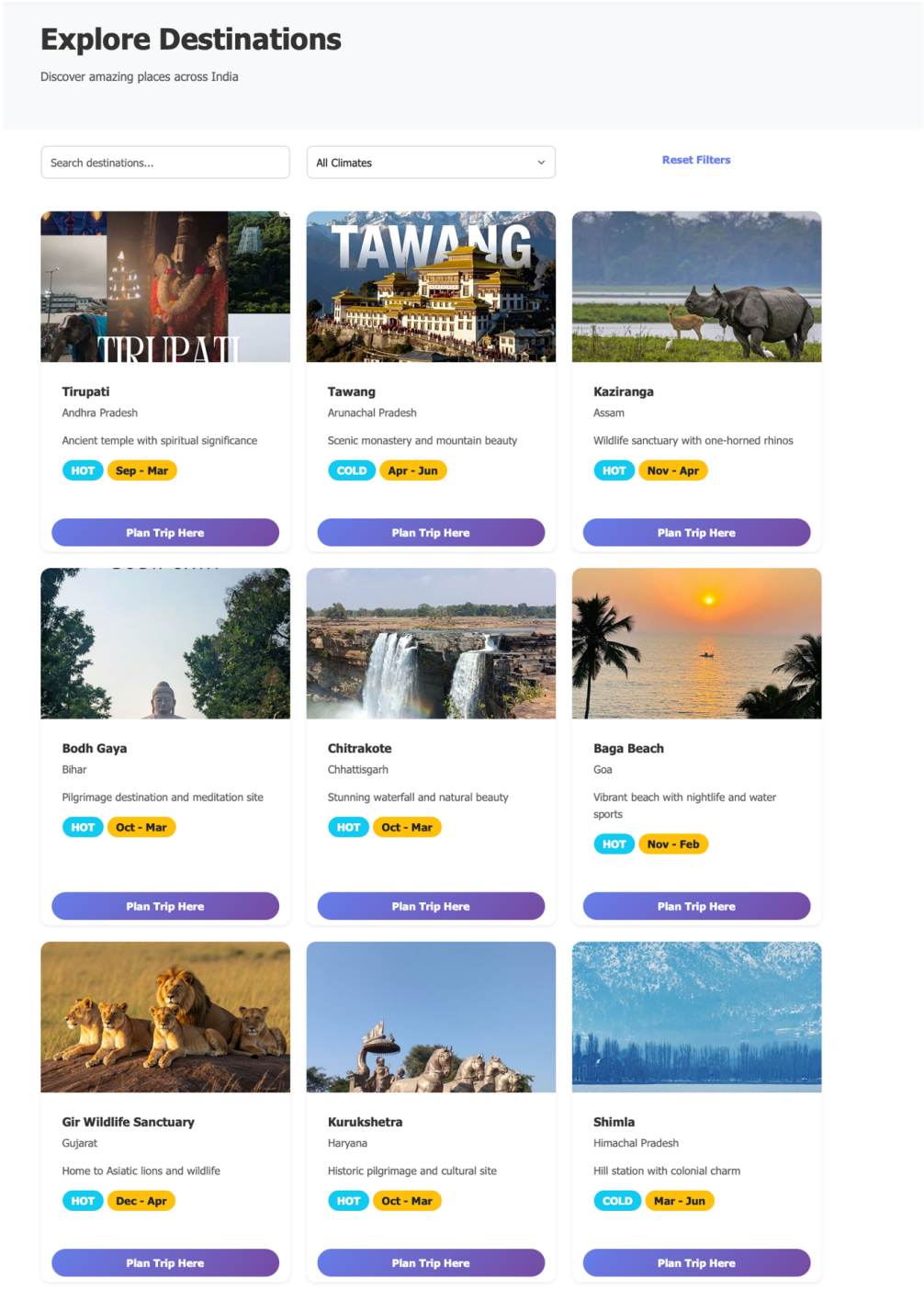


Figure 6.6: Destinations Explorer Interface

6.2.3 Trip Planning Form

The trip planning form acts as the primary interaction point for recommendation generation. The form securely collects:

- **Budget**: Numeric input in INR with minimum value validation
- **Travel Days**: Integer selector (1–30 days)
- **Climate Preference**: Dropdown selection (Hot, Cold, Moderate)
- **Travel Type**: Dropdown selection (Adventure, Relaxation, Cultural, Family)

Interactive dropdowns and client-side validation improve user input accuracy and usability. Form submission is handled asynchronously via the JavaScript Fetch API, providing a seamless user experience without full page reloads.

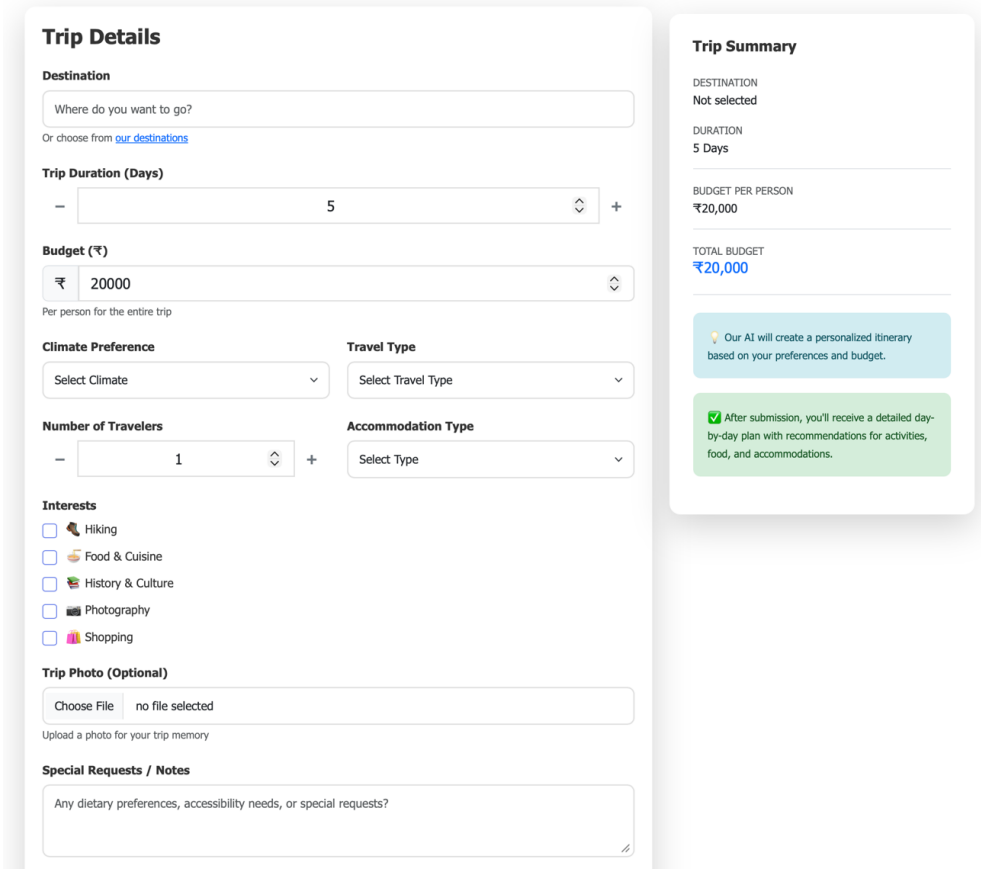


Figure 6.7: Trip Planning Form Interface

6.2.4 Dashboard (My Trips)

The dashboard provides personalized trip management features including:

- **Saved Itineraries**: Tabular display of all saved trip plans with destination images

- **Travel History**: Date-stamped records with budget and travel type information
- **Soft-Delete Functionality**: Trips moved to a "Recently Deleted" section rather than permanently erased
- **Trip Restoration**: Deleted trips can be recycled back to the active list

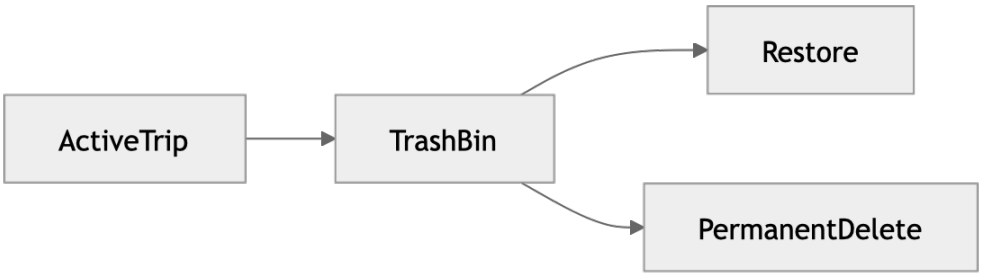


Figure 6.8: Soft Delete Workflow

The soft-delete mechanism prevents accidental data loss by setting the `is_deleted` flag to `True` and recording the deletion timestamp in `deleted_at`. The active trips query filters for records where `is_deleted=False`.

6.2.5 Results View

The results interface displays the complete trip plan generated by the AI recommendation engine:

- **Predicted Destination**: Name and destination image
- **Budget Summaries**: Total budget with percentage distribution across categories
- **Daily Itinerary Schedules**: Day-by-day activity plan
- **Activity Suggestions**: Recommended attractions and experiences
- **Travel Guidance**: Practical tips for the recommended destination

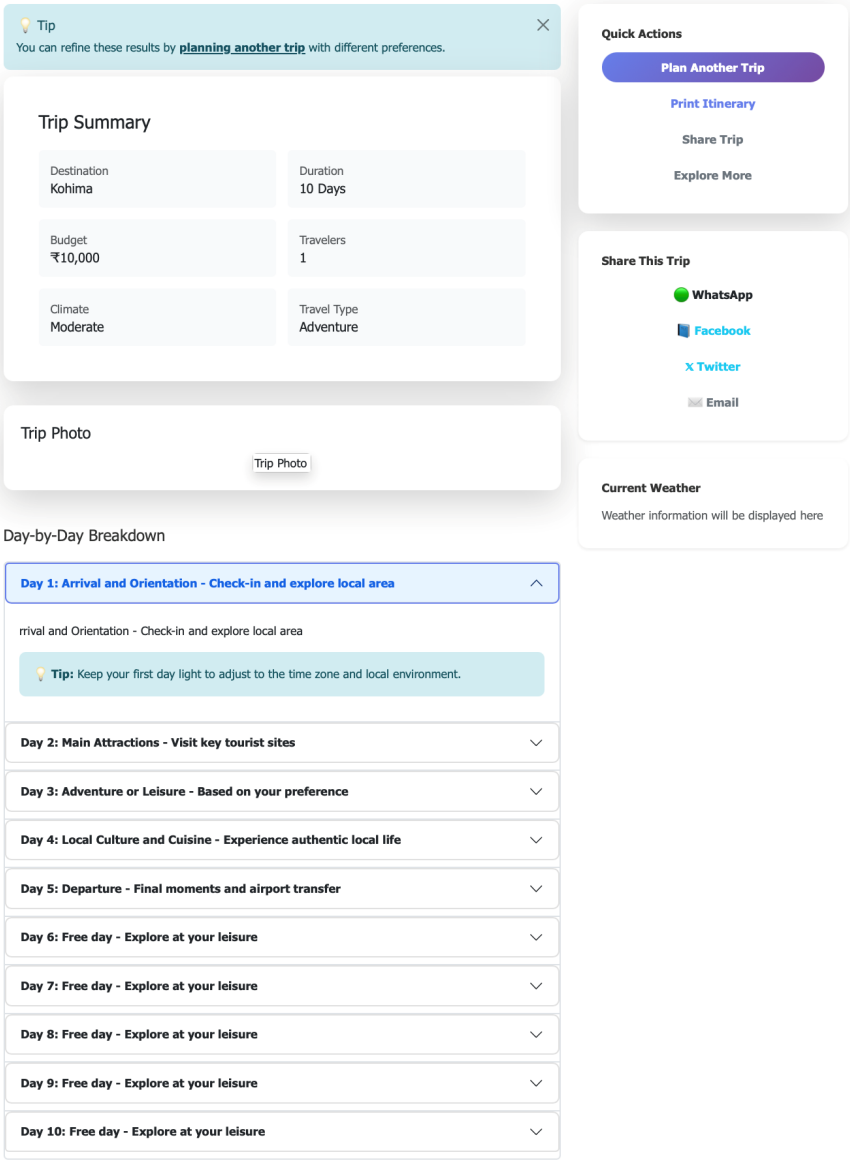


Figure 6.9: Results View - Destination Recommendation and Cost Breakdown

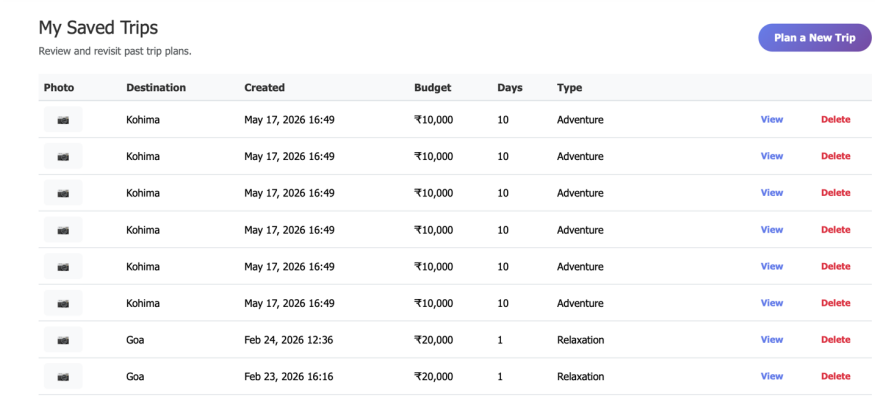


Figure 6.10: Results View - Day-wise Itinerary

The application visualizes budget distribution using percentages and organized layout structures to improve readability and user understanding. The interface is designed to create an immersive and professional travel planning experience while maintaining performance and responsiveness across all supported devices.

Chapter 7

Testing and Evaluation

7.1 Unit Testing

Unit testing focuses on validating individual modules and functions independently to ensure they behave as expected. In the AI-Based Trip Planner Web Application, unit tests are implemented to verify the correctness of critical backend logic, machine learning utilities, itinerary generation mechanisms, and data transformation functions. The testing process is primarily conducted using Python's built-in `unittest` framework and Django's testing utilities.

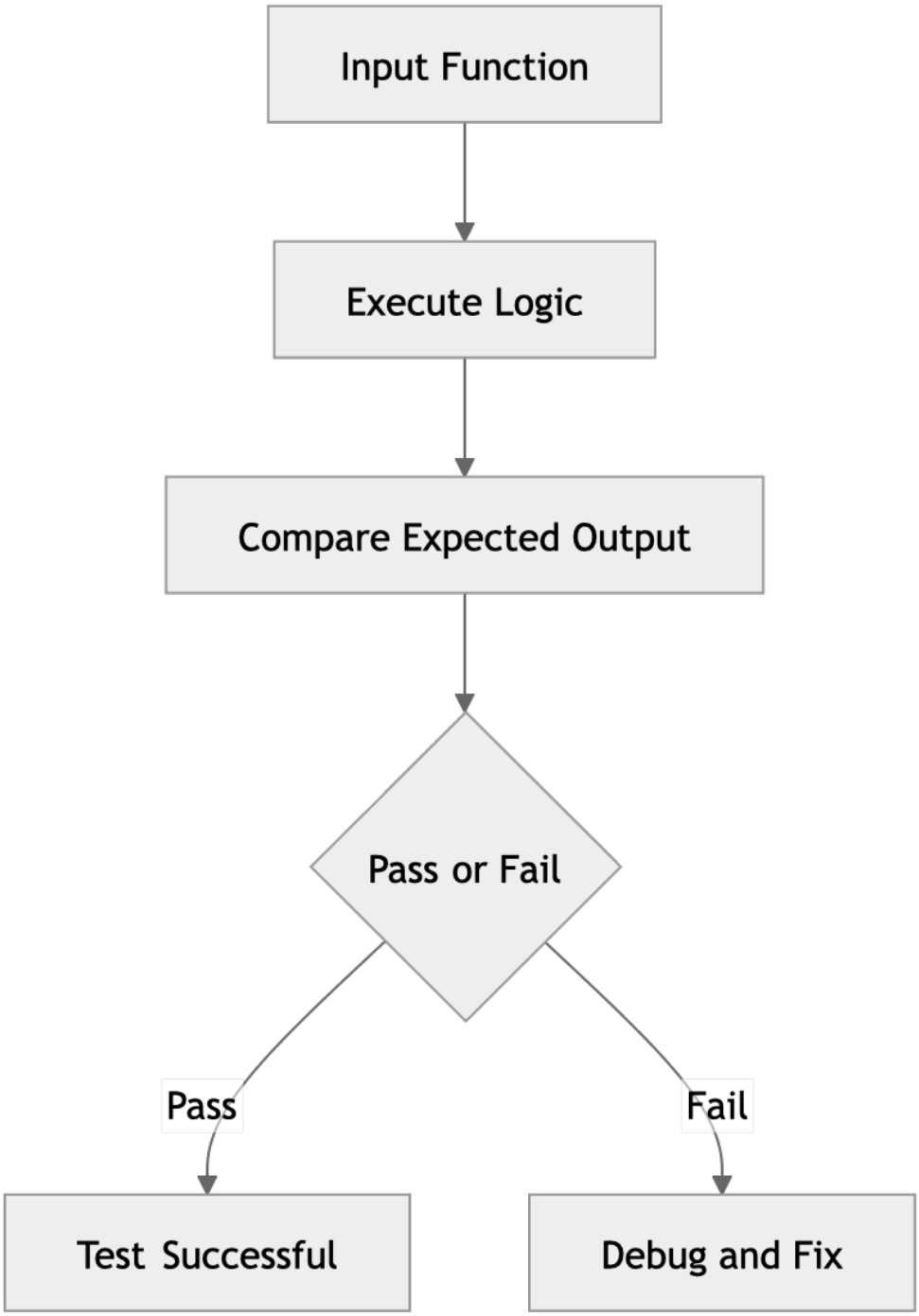


Figure 7.1: Unit Testing Workflow

The major objectives of unit testing include:

- Detecting logical errors in isolated components
- Validating data transformation functions
- Ensuring accurate cost calculations

- Verifying machine learning preprocessing
- Maintaining code reliability during updates

7.1.1 Testing the Dynamic Itinerary Generator

The `generate_trip_plan` module is one of the most critical components of the application. Unit tests are designed to validate:

- Budget calculations and cost multipliers
- Itinerary structures and day allocation logic
- Percentage distributions for budget breakdown

One important validation ensures that the generated budget breakdown percentages always total exactly 100%:

- Accommodation = 40%
- Food = 25%
- Transportation = 20%
- Activities = 15%
- **Total = 100%**

This test prevents logical inconsistencies in financial reporting and ensures accurate travel budgeting.

7.1.2 Testing LabelEncoder Transformations

Unit tests verify that LabelEncoder transformations are bijective – meaning the encoding and inverse-encoding operations are consistent. Tests confirm:

- `climate_encoder.transform(['hot'])` returns a consistent integer value
- `climate_encoder.inverse_transform([value])` returns the original string correctly
- All 36 destinations are correctly encoded and decodable without collisions

- Climate values (hot, cold, moderate) map to exactly 3 distinct classes
- Travel type values map to exactly 4 distinct classes

```python
import unittest
from sklearn.preprocessing import LabelEncoder

class TestLabelEncoder(unittest.TestCase):
    def setUp(self):
        self.encoder = LabelEncoder()
        self.encoder.fit(['hot', 'cold', 'moderate'])

    def test_transform(self):
        result = self.encoder.transform(['hot'])
        self.assertIsInstance(result[0], (int, ))

    def test_inverse_transform(self):
        encoded = self.encoder.transform(['cold'])
        decoded = self.encoder.inverse_transform(encoded)
        self.assertEqual(decoded[0], 'cold')

    def test_all_classes_covered(self):
        self.assertEqual(len(self.encoder.classes_), 3)
```

Listing 7.1: Unit Test for LabelEncoder

7.1.3 Testing Database Models

Unit tests for the `Trip` model verify:

- Trip objects are created successfully with valid field values
- Default values are correctly applied (e.g., `is_deleted=False`)
- Soft-delete behavior sets `is_deleted=True` and records `deleted_at`
- Trip restoration correctly resets `is_deleted=False`
- JSON plan data is serialized and deserialized correctly

7.2 Integration Testing

Integration testing validates the interaction between system components to ensure end-to-end functionality operates correctly. The AI-Based Trip Planner

Web Application undergoes integration testing across API endpoints, database interactions, and authentication flows.

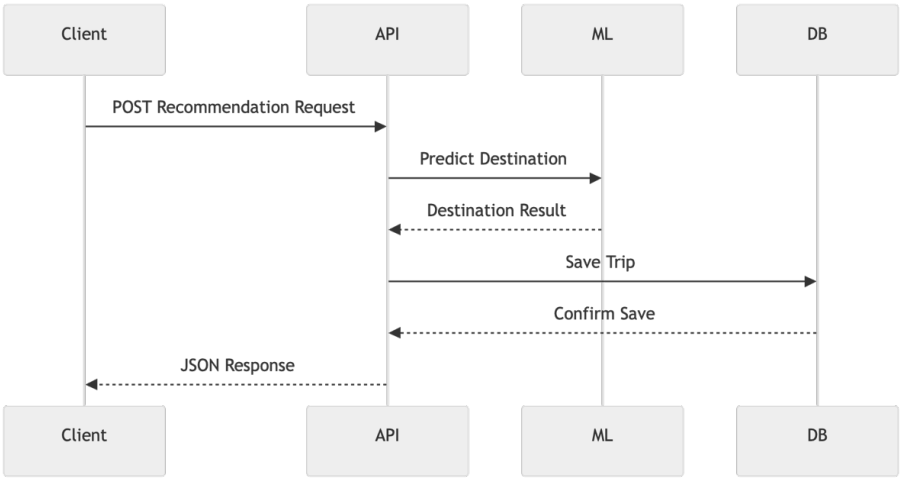


Figure 7.2: Integration Testing Architecture

7.2.1 API Endpoint Testing

API integration tests use Django's `APITestCase` to send simulated HTTP requests to the recommendation endpoint and verify:

- HTTP 200 OK response for valid input payloads
- HTTP 400 Bad Request response for invalid or missing fields
- Returned JSON contains required keys: `destination`, `budget`, `itinerary`
- Destination in response is a valid known destination string
- Cost breakdown values sum correctly to the submitted budget

7.2.2 Database Interaction Testing

Database integration tests verify:

- Trip records are saved correctly after a successful recommendation API call
- Soft-deleted trips do not appear in the active trips query
- Recycled trips reappear in the active trips list
- Trip counts increase correctly per user session

7.2.3 Authentication and Session Testing

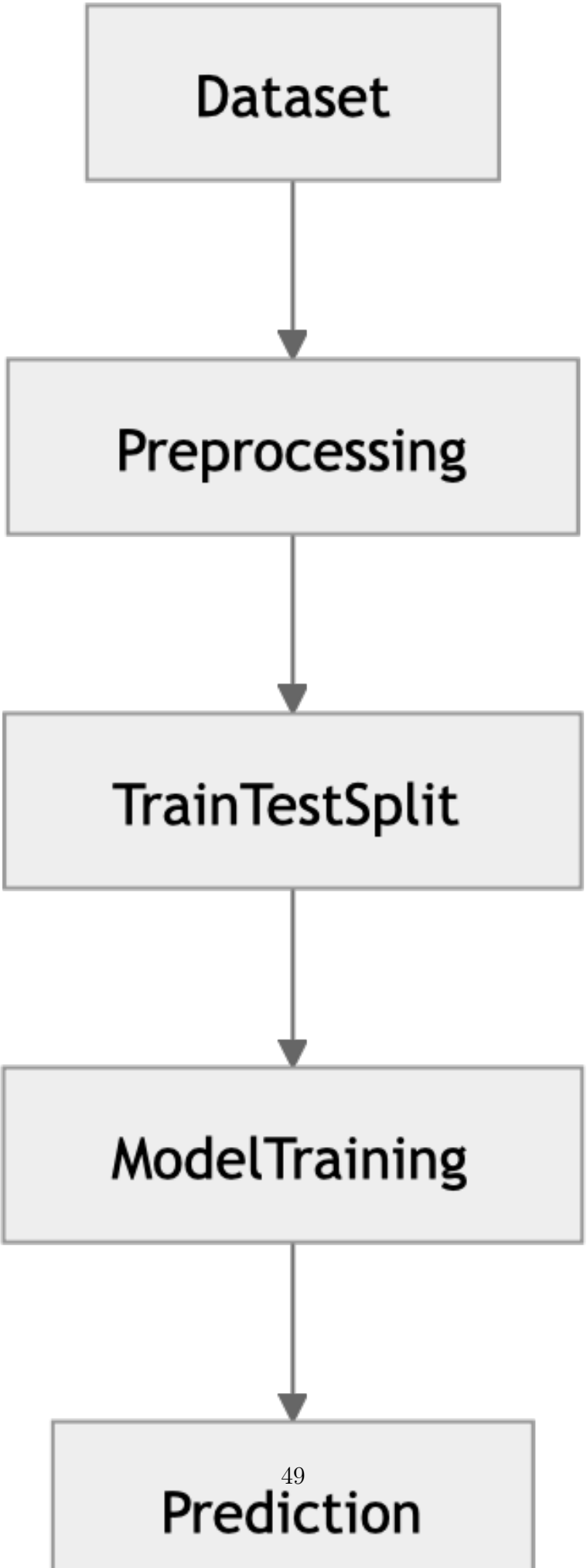
Authentication integration tests verify:

- Unauthenticated users are redirected to the login page for protected routes
- Valid credentials create an active session
- Invalid credentials return appropriate error messages
- Logout correctly clears the session
- CSRF token validation prevents unauthorized form submissions

7.3 Model Accuracy and Performance

7.3.1 Accuracy Evaluation

The Decision Tree classifier is evaluated on a held-out test subset (20% of the synthetic dataset) using the standard accuracy metric:


Dataset
Preprocessing
TrainTestSplit
ModelTraining
Prediction

The model achieves high classification accuracy due to:

- **Consistent synthetic data**: Destinations are generated with strict budget ranges, climate types, and travel categories, creating highly separable class boundaries.
- **Optimal tree depth**: The maximum depth of 8 provides sufficient complexity to distinguish between 36 destination classes without overfitting.
- **Feature effectiveness**: The four input features (budget, days, climate, travel type) provide strong discriminative information for destination classification.
- **Controlled randomness**: The `random_state=42` parameter ensures reproducible results across different training runs.

7.3.2 Inference Speed and Runtime Performance

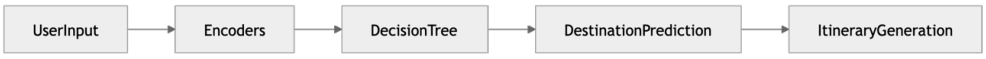


Figure 7.4: Prediction Runtime Workflow

Inference performance characteristics:

- **Model loading time**: Under 100ms at application startup (Joblib deserialization)
- **Per-request prediction latency**: Under 5ms for a single prediction
- **Concurrent request handling**: Django's WSGI server handles multiple concurrent requests with negligible ML overhead

7.3.3 System Scalability Considerations

The system is designed to support scaling through the following strategies:

- **Stateless ML inference**: The serialized model is loaded once at startup and shared across all requests without reloading.
- **Database connection pooling**: Django's ORM supports connection pooling for high-traffic scenarios.

- **Static file serving**: CSS, JavaScript, and images can be offloaded to a CDN for improved global delivery.
- **Horizontal scaling**: The application can be deployed across multiple workers using Gunicorn or uWSGI behind a load balancer.
- **Database migration**: SQLite3 can be replaced with PostgreSQL for production deployments requiring concurrent write support.

Chapter 8

Conclusion and Future Scope

8.1 Summary of Contributions

This project successfully demonstrates the integration of artificial intelligence, machine learning, and full-stack web development to create a functional, user-centric AI-Based Trip Planner Web Application. The key contributions of this work are summarized below.

8.1.1 Intelligent Destination Recommendation System

The project develops and deploys a machine learning-based destination recommendation engine using the **Decision Tree Classifier** from Scikit-learn. The system accepts user-specified travel constraints (budget, days, climate, travel type) and predicts the most suitable travel destination from a curated list of 36 Indian destinations. The recommendation engine achieves high classification accuracy due to the carefully designed synthetic dataset and optimized tree depth configuration. The trained model is serialized using Joblib for fast, real-time inference at application runtime [3].

8.1.2 Dynamic Itinerary Generation

The application implements an automated itinerary generation system that produces structured, day-wise travel schedules following destination prediction. The engine dynamically adapts schedules to match the user's specified trip duration, distributes costs across standard travel expense categories (accommodation, food, transportation, activities), and outputs JSON-structured trip data for frontend rendering. This feature significantly reduces the planning effort required from users.

8.1.3 Full-Stack Web Application Development

The complete web application is built using **Django** [4] following the MVT architectural pattern. The backend implements:

- Secure user authentication with Django's built-in `AbstractUser` system
- RESTful API development using Django REST Framework
- Modular view and URL routing structures
- ORM-based database management with SQLite3

8.1.4 User-Centric Features

The system provides personalized trip management capabilities that enhance user experience:

- **Trip Dashboard**: Tabular display of all saved trip plans with destination images and metadata
- **Soft-Delete Mechanism**: Prevents permanent data loss by moving deleted trips to a recoverable trash state
- **Trip Restoration**: Users can restore previously deleted trips from the dashboard
- **Responsive Design**: Cross-device compatibility across desktop, tablet, and mobile platforms

8.1.5 Real-World Applicability

The application demonstrates practical applicability for Indian travel planning:

- Covers all major Indian states and union territories through curated destination data
- Uses INR-based budget ranges that reflect realistic Indian tourism costs
- Categorizes travel types relevant to the Indian tourism market
- Provides culturally relevant itinerary suggestions for each destination

8.2 Future Enhancements

While the current implementation successfully demonstrates the core objectives of the project, several enhancements can significantly extend its capabilities and real-world utility.

8.2.1 Global Dataset Expansion

The current dataset covers 36 Indian travel destinations. Future work includes expanding the recommendation system to cover international destinations:

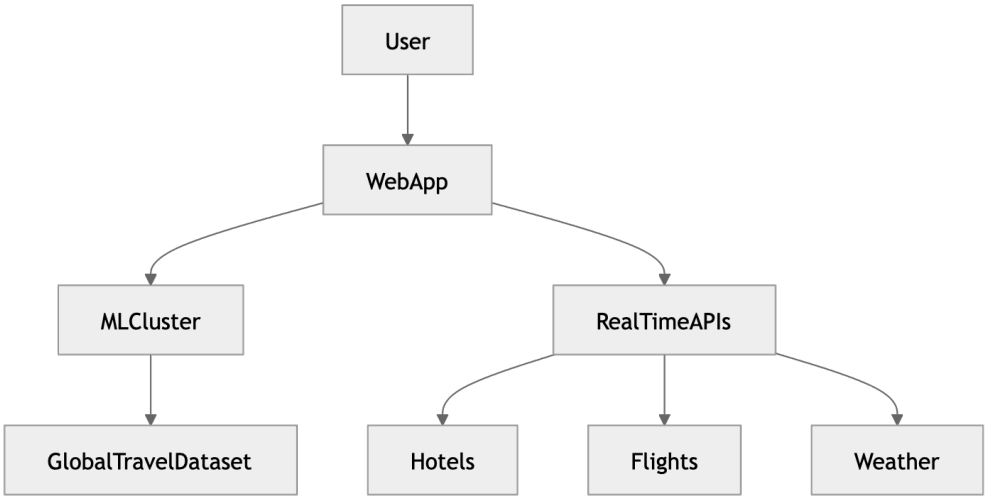


Figure 8.1: Future Scalable Architecture

Global dataset expansion would involve:

- Integration with publicly available travel datasets (e.g., Kaggle tourism datasets)
- Multi-currency budget support with dynamic currency conversion
- Multilingual destination descriptions and itinerary content
- Region-specific travel regulations and visa requirement information

8.2.2 Real-Time Price API Integration

The current system uses static budget ranges derived from the synthetic dataset. Future enhancements will incorporate real-time price data through third-party API integrations:

- **Flight pricing APIs**: Skyscanner or Amadeus APIs for real-time flight cost data
- **Hotel pricing APIs**: Booking.com or Expedia APIs for live accommodation rates
- **Weather APIs**: OpenWeatherMap API for current and forecasted weather conditions
- **Currency exchange APIs**: Live exchange rates for international budget planning

Real-time data integration would make budget estimates significantly more accurate and relevant to actual market conditions.

8.2.3 Collaborative Filtering Recommendations

The current recommendation engine uses a supervised classification approach. Future work will explore hybrid recommendation models combining:

- **Collaborative Filtering**: Recommendations based on the travel preferences of similar users
- **Matrix Factorization**: Latent factor models for capturing hidden user-destination relationships
- **Neural Collaborative Filtering**: Deep learning models for non-linear preference modeling

As the user base grows, collaborative filtering will enable increasingly personalized recommendations based on collective user behavior.

8.2.4 Export and Sharing Features

Future versions of the application will include trip plan export and social sharing capabilities:

- PDF export of complete itineraries using ReportLab or WeasyPrint
- Email delivery of trip plans to registered user email addresses
- Social media sharing links for Instagram, WhatsApp, and Facebook
- Calendar integration for exporting itinerary dates to Google Calendar or ICS format

8.2.5 Progressive Web Application (PWA) Support

Converting the web application into a Progressive Web Application will extend its reach to mobile users:

- Offline access to saved trip itineraries using service workers
- Push notifications for travel reminders and updates
- Home screen installation on Android and iOS devices
- Improved mobile performance through caching strategies

PWA support would enable the application to function as a native-like mobile travel companion without requiring App Store or Play Store distribution.

REFERENCES

[1] D. Buhalis and R. Law, "Progress in information technology and tourism management: 20 years on and 10 years after the Internet – The state of eTourism research," *Tourism Management*, vol. 29, no. 4, pp. 609–623, 2008.

[2] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, 2nd. New York, NY: Springer, 2015.

[3] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] Django Software Foundation. "Django documentation." Django Software Foundation. Retrieved June 2025. (2026), [Online]. Available: `https://docs.djangoproject.com/` (visited on 06/01/2025).

[5] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 51–56.

Appendix A

Source Code

This appendix contains the complete source code for the AI-Based Trip Planner Web Application, including the machine learning training pipeline, the Django views and URL configurations, database models, serializers, and the responsive HTML templates.

A.1 Machine Learning and Backend Logic

A.1.1 `Train.model.py`

```
import argparse
import random
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import joblib
import os


BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_PATH = os.path.join(BASE_DIR, "data", "trip_dataset.csv")


# ------------------------------
# 1. DESTINATION RULES (used only for synthetic fallback)
# All 36 Indian States and Union Territories with representative tourist destinations
# ------------------------------
DESTINATIONS = [
    ("Tirupati", "hot", "cultural", 15000, 35000),
    ("Tawang", "cold", "adventure", 25000, 50000),
```

```
    ("Kaziranga", "hot", "adventure", 20000, 45000),
    ("Bodh Gaya", "hot", "religious", 12000, 30000),
    ("Chitrakote", "hot", "relaxation", 10000, 25000),
    ("Baga Beach", "hot", "relaxation", 15000, 40000),
    ("Gir Wildlife Sanctuary", "hot", "adventure", 18000, 40000),
    ("Kurukshetra", "hot", "cultural", 10000, 25000),
    ("Shimla", "cold", "family", 15000, 35000),
    ("Ranchi", "moderate", "family", 12000, 30000),
    ("Hampi", "moderate", "cultural", 12000, 30000),
    ("Alleppey Backwaters", "hot", "relaxation", 20000, 45000),
    ("Khajuraho", "hot", "cultural", 15000, 35000),
    ("Ajanta Caves", "hot", "cultural", 12000, 30000),
    ("Loktak Lake", "moderate", "relaxation", 12000, 28000),
    ("Cherrapunji", "moderate", "adventure", 15000, 35000),
    ("Aizawl", "moderate", "cultural", 12000, 28000),
    ("Kohima", "moderate", "cultural", 12000, 28000),
    ("Puri", "hot", "religious", 12000, 30000),
    ("Golden Temple", "hot", "religious", 10000, 25000),
    ("Jaipur", "hot", "cultural", 12000, 35000),
    ("Gangtok", "moderate", "family", 15000, 35000),
    ("Mahabalipuram", "hot", "cultural", 12000, 30000),
    ("Hyderabad", "hot", "cultural", 12000, 30000),
    ("Ujjayanta Palace", "hot", "cultural", 10000, 25000),
    ("Agra", "hot", "cultural", 12000, 35000),
    ("Rishikesh", "moderate", "adventure", 10000, 28000),
    ("Darjeeling", "cold", "family", 15000, 35000),
    ("Port Blair", "hot", "relaxation", 20000, 45000),
    ("Rock Garden", "moderate", "family", 10000, 25000),
    ("Daman Beach", "hot", "relaxation", 12000, 30000),
    ("Srinagar", "cold", "relaxation", 20000, 45000),
    ("Pangong Lake", "cold", "adventure", 30000, 60000),
    ("Minicoy Island", "hot", "relaxation", 25000, 50000),
    ("Pondicherry", "hot", "relaxation", 12000, 30000),
    ("India Gate", "hot", "cultural", 10000, 30000),
]


# ------------------------------
# 2. DATASET LOADING / GENERATION
# ------------------------------


if os.path.exists(DATA_PATH):
    # load a dataset exported from Kaggle (or any other source) into the training folder
    data = pd.read_csv(DATA_PATH)
```

```
    # basic cleaning: ensure important columns are present and numeric where expected
    expected_cols = {"budget", "days", "climate", "travel_type", "destination"}
    if not expected_cols.issubset(set(data.columns)):
        raise ValueError(f"Dataset at {DATA_PATH} missing required columns: {expected_cols - set(data.columns)}")
    # coerce numeric fields and drop rows with missing values
    data["budget"] = pd.to_numeric(data["budget"], errors="coerce")
    data["days"] = pd.to_numeric(data["days"], errors="coerce")
    data = data.dropna(subset=["budget", "days", "climate", "travel_type", "destination"])

    # remove image_url column if it exists
    if "image_url" in data.columns:
        data = data.drop(columns=["image_url"])


    print(f"Loaded dataset from {DATA_PATH}:", data.shape)
else:

    rows = []


    for _ in range(5000):
        destination, climate, travel_type, min_budget, max_budget = random.choice(DESTINATIONS)


        budget = random.randint(min_budget, max_budget)
        days = random.randint(2, 7)


        rows.append([
            budget,
            days,
            climate,
            travel_type,
            destination
        ])


    data = pd.DataFrame(
        rows,
        columns=["budget", "days", "climate", "travel_type", "destination"]
    )
```

```
107
108
109     print("Synthetic dataset generated:", data.shape)
110
111
112 # ------------------------------
113 # 3. ENCODING
114 # ------------------------------
115 le_climate = LabelEncoder()
116 le_travel = LabelEncoder()
117 le_destination = LabelEncoder()
118
119
120 data["climate"] = le_climate.fit_transform(data["climate"])
121 data["travel_type"] = le_travel.fit_transform(data["travel_type"])
122 data["destination"] = le_destination.fit_transform(data["destination"
        ])
123
124
125 X = data[["budget", "days", "climate", "travel_type"]]
126 y = data["destination"]
127
128
129 # ------------------------------
130 # 4. TRAIN MODEL
131 # ------------------------------
132 model = DecisionTreeClassifier(
133     max_depth=8,
134     random_state=42
135 )
136
137
138 model.fit(X, y)
139
140
141 # ------------------------------
142 # 5. SAVE MODEL FILES
143 # ------------------------------
144 joblib.dump(model, os.path.join(BASE_DIR, "trip_model.pkl"))
145 joblib.dump(le_climate, os.path.join(BASE_DIR, "climate_encoder.pkl")
        )
146 joblib.dump(le_travel, os.path.join(BASE_DIR, "travel_encoder.pkl"))
147 joblib.dump(le_destination, os.path.join(BASE_DIR, "
        destination_encoder.pkl"))
148
149
```

```
150 print(f"Model training complete. Model saved for {le_destination.
    classes_.size} destinations.")
```

Listing A.1: Train.model.py

A.1.2 service.py

```
1  import os
2  import joblib
3  import numpy as np
4  from django.conf import settings
5
6
7  # Load model paths
8  ML_FOLDER = os.path.join(settings.BASE_DIR, "trips", "ml")
9
10
11 model = joblib.load(os.path.join(ML_FOLDER, "trip_model.pkl"))
12 climate_encoder = joblib.load(os.path.join(ML_FOLDER, "
    climate_encoder.pkl"))
13 travel_encoder = joblib.load(os.path.join(ML_FOLDER, "travel_encoder.
    pkl"))
14 destination_encoder = joblib.load(os.path.join(ML_FOLDER, "
    destination_encoder.pkl"))
15
16
17 def predict_destination(budget, days, climate, travel_type):
18     # Encode categorical values
19     climate_encoded = climate_encoder.transform([climate])[0]
20     travel_encoded = travel_encoder.transform([travel_type])[0]
21
22
23     # Prepare input
24     input_data = np.array([[budget, days, climate_encoded,
    travel_encoded]])
25
26
27     # Predict
28     prediction = model.predict(input_data)
29
30
31     # Decode result
32     destination = destination_encoder.inverse_transform(prediction)
    [0]
33
34
```

```
35      return destination
```

Listing A.2: service.py

A.1.3 urls.py

```
1  from django.urls import path
2  from .views import (
3      TripRecommendationView,
4      home_page,
5      destinations_page,
6      plan_trip_page,
7      trip_results_page,
8      my_trips_page,
9      trip_detail_page,
10     delete_trip,
11     recycle_trip,
12     login_page,
13     logout_page,
14     register_page,
15 )
16
17
18 urlpatterns = [
19     path('', home_page, name='home'),
20     path('login/', login_page, name='login'),
21     path('logout/', logout_page, name='logout'),
22     path('register/', register_page, name='register'),
23     path('destinations/', destinations_page, name='destinations'),
24     path('plan-trip/', plan_trip_page, name='plan-trip'),
25     path('results/', trip_results_page, name='trip-results'),
26     path('my-trips/', my_trips_page, name='my-trips'),
27     path('trips/<int:trip_id>/', trip_detail_page, name='trip-detail'
       ),
28     path('trips/<int:trip_id>/delete/', delete_trip, name='delete-
       trip'),
29     path('trips/<int:trip_id>/recycle/', recycle_trip, name='recycle-
       trip'),
30     path('recommend/', TripRecommendationView.as_view(), name='
       recommend-trip'),
31 ]
```

Listing A.3: urls.py

A.1.4 App.py

```python
from django.apps import AppConfig


class TripsConfig(AppConfig):
    name = 'trips'
```

Listing A.4: App.py

A.1.5 serializers.py

```python
from rest_framework import serializers
from .models import Trip


class TripSerializer(serializers.ModelSerializer):
    class Meta:
        model = Trip
        fields = "__all__"
        read_only_fields = ["itinerary", "created_at", "plan_data"]


class TripRequestSerializer(serializers.Serializer):
    days = serializers.IntegerField()
    budget = serializers.FloatField()
    climate = serializers.CharField()
    travel_type = serializers.CharField()
    travelers = serializers.IntegerField(required=False, default=1)
    accommodation = serializers.CharField(required=False)
    interests = serializers.ListField(child=serializers.CharField(),
    required=False, default=list)
    special_requests = serializers.CharField(required=False,
    allow_blank=True)
    image = serializers.ImageField(required=False, allow_null=True)
```

Listing A.5: serializers.py

A.1.6 models.py

```python
from django.db import models
from django.core.files.storage import default_storage
from django.utils import timezone


class Trip(models.Model):
    CLIMATE_CHOICES = [
```

```
        ('hot', 'Hot'),
        ('cold', 'Cold'),
        ('moderate', 'Moderate'),
    ]


    TRAVEL_TYPE_CHOICES = [
        ('adventure', 'Adventure'),
        ('relaxation', 'Relaxation'),
        ('cultural', 'Cultural'),
        ('family', 'Family'),
    ]


    destination = models.CharField(max_length=255, default='Unknown')
    budget = models.IntegerField(default=0)
    days = models.IntegerField(default=1)
    climate = models.CharField(max_length=20, choices=CLIMATE_CHOICES
, default='moderate', null=True, blank=True)
    travel_type = models.CharField(max_length=20, choices=
TRAVEL_TYPE_CHOICES, default='relaxation', null=True, blank=True)
    itinerary = models.TextField(blank=True)


    # Full structured trip plan (used to render the results page and
export)
    plan_data = models.JSONField(default=dict, blank=True)


    # Trip image (optional)
    image = models.ImageField(upload_to='trip_images/', blank=True,
null=True)


    # Soft delete field
    is_deleted = models.BooleanField(default=False)
    deleted_at = models.DateTimeField(null=True, blank=True)


    created_at = models.DateTimeField(auto_now_add=True)


    def __str__(self):
        return f"{self.destination} - {self.budget}"


    def delete(self, *args, **kwargs):
```

```
        # Soft delete: mark as deleted instead of actually deleting
        self.is_deleted = True
        self.deleted_at = timezone.now()
        self.save()


    def recycle(self):
        # Restore a soft-deleted trip
        self.is_deleted = False
        self.deleted_at = None
        self.save()


    def hard_delete(self, *args, **kwargs):
        # Actually delete the trip and its image file
        if self.image:
            default_storage.delete(self.image.name)
        super().delete(*args, **kwargs)
```

Listing A.6: models.py

A.1.7 views.py

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.decorators import api_view
from django.views.decorators.csrf import csrf_exempt
from django.utils.decorators import method_decorator
from .serializers import TripRequestSerializer
from .services.ai_engine import predict_destination,
    generate_trip_plan
from .models import Trip
import json
import os
from urllib.parse import unquote


from django.conf import settings
from django.shortcuts import render, redirect, get_object_or_404
from django.urls import reverse
from django.contrib import messages
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User
from django.contrib.auth.decorators import login_required

```

```
23
24 def get_destination_image_url(destination, default_query):
25     """Return a local static image if available, otherwise fallback
       to Unsplash."""
26     filename = f"{destination.lower().replace(' ', '_')}.jpg"
27     local_path = os.path.join(settings.BASE_DIR, "static", "images",
       filename)
28     if os.path.exists(local_path):
29         static_url = settings.STATIC_URL
30         if not static_url.startswith('/'):
31             static_url = '/' + static_url
32         static_url = static_url.rstrip('/') + '/'
33         return f"{static_url}images/{filename}"
34     return f"https://source.unsplash.com/600x400/?{default_query}"
35
36
37 def login_page(request):
38     """Handle user login"""
39     if request.method == 'POST':
40         username = request.POST.get('username')
41         password = request.POST.get('password')
42         user = authenticate(request, username=username, password=
       password)
43         if user is not None:
44             login(request, user)
45             messages.success(request, f'Welcome back, {username}!')
46             return redirect('home')
47         else:
48             messages.error(request, 'Invalid username or password.')
49     return render(request, 'login.html')
50
51
52 def register_page(request):
53     """Handle user registration"""
54     if request.method == 'POST':
55         username = request.POST.get('username')
56         email = request.POST.get('email')
57         password = request.POST.get('password')
58         password_confirm = request.POST.get('password_confirm')
59
60         if password != password_confirm:
61             messages.error(request, 'Passwords do not match.')
62             return render(request, 'register.html')
63
64         if User.objects.filter(username=username).exists():
65             messages.error(request, 'Username already exists.')
66             return render(request, 'register.html')
```

```
67
68          if User.objects.filter(email=email).exists():
69              messages.error(request, 'Email already exists.')
70              return render(request, 'register.html')
71
72          user = User.objects.create_user(username=username, email=
    email, password=password)
73          login(request, user)
74          messages.success(request, f'Welcome, {username}! Your account
     has been created.')
75          return redirect('home')
76
77      return render(request, 'register.html')
78
79
80  def logout_page(request):
81      """Handle user logout"""
82      logout(request)
83      messages.success(request, 'You have been logged out successfully.
    ')
84      return redirect('login')
85
86
87  def home_page(request):
88      """Show landing page for anonymous users, home page for
    authenticated users"""
89      if not request.user.is_authenticated:
90          return render(request, "landing.html")
91      return render(request, "home.html")
92
93
94  @login_required(login_url='login')
95  def destinations_page(request):
96      """Display all available destinations from trained model"""
97      destinations_data = [
98          {'name': 'Tirupati', 'image_file': 'tirupati.jpg', 'state': '
    Andhra Pradesh', 'climate': 'hot', 'best_time': 'Sep - Mar', '
    description': 'Ancient temple with spiritual significance'},
99          {'name': 'Tawang', 'image_file': 'tawang.jpg', 'state': '
    Arunachal Pradesh', 'climate': 'cold', 'best_time': 'Apr - Jun', '
    description': 'Scenic monastery and mountain beauty'},
100         {'name': 'Kaziranga', 'image_file': 'kaziranga.jpg', 'state':
     'Assam', 'climate': 'hot', 'best_time': 'Nov - Apr', 'description
    ': 'Wildlife sanctuary with one-horned rhinos'},
101         {'name': 'Bodh Gaya', 'image_file': 'bodh_gaya.jpg', 'state':
     'Bihar', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description
    ': 'Pilgrimage destination and meditation site'},
```

```
102         {'name': 'Chitrakote', 'image_file': 'chitrakote.jpg', 'state
    ': 'Chhattisgarh', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'Stunning waterfall and natural beauty'},
103         {'name': 'Baga Beach', 'image_file': 'baga_beach.jpg', 'state
    ': 'Goa', 'climate': 'hot', 'best_time': 'Nov - Feb', 'description
    ': 'Vibrant beach with nightlife and water sports'},
104         {'name': 'Gir Wildlife Sanctuary', 'image_file': '
    gir_wildlife_sanctuary.jpg', 'state': 'Gujarat', 'climate': 'hot',
     'best_time': 'Dec - Apr', 'description': 'Home to Asiatic lions
    and wildlife'},
105         {'name': 'Kurukshetra', 'image_file': 'kurukshetra.jpg', '
    state': 'Haryana', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'Historic pilgrimage and cultural site'},
106         {'name': 'Shimla', 'image_file': 'shimla.jpg', 'state': '
    Himachal Pradesh', 'climate': 'cold', 'best_time': 'Mar - Jun', '
    description': 'Hill station with colonial charm'},
107         {'name': 'Ranchi', 'image_file': 'ranchi.jpg', 'state': '
    Jharkhand', 'climate': 'moderate', 'best_time': 'Oct - Mar', '
    description': 'Waterfalls and natural parks'},
108         {'name': 'Hampi', 'image_file': 'hampi.jpg', 'state': '
    Karnataka', 'climate': 'moderate', 'best_time': 'Oct - Feb', '
    description': 'UNESCO heritage ruins and architecture'},
109         {'name': 'Alleppey Backwaters', 'image_file': '
    alleppey_backwaters.jpg', 'state': 'Kerala', 'climate': 'hot', '
    best_time': 'Sep - Mar', 'description': 'Houseboat cruises and
    backwater beauty'},
110         {'name': 'Khajuraho', 'image_file': 'khajuraho.jpg', 'state':
     'Madhya Pradesh', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'Ancient temples with intricate carvings'},
111         {'name': 'Ajanta Caves', 'image_file': 'ajanta_caves.jpg', '
    state': 'Maharashtra', 'climate': 'hot', 'best_time': 'Nov - Mar',
     'description': 'Ancient Buddhist cave complex'},
112         {'name': 'Loktak Lake', 'image_file': 'loktak_lake.jpg', '
    state': 'Manipur', 'climate': 'moderate', 'best_time': 'Oct - Mar'
    , 'description': 'Unique floating lake with natural beauty'},
113         {'name': 'Cherrapunji', 'image_file': 'cherrapunji.jpg', '
    state': 'Meghalaya', 'climate': 'moderate', 'best_time': 'Oct -
    May', 'description': 'Waterfalls and green landscapes'},
114         {'name': 'Aizawl', 'image_file': 'aizawl.jpg', 'state': '
    Mizoram', 'climate': 'moderate', 'best_time': 'Oct - May', '
    description': 'Hill city with cultural richness'},
115         {'name': 'Kohima', 'image_file': 'kohima.jpg', 'state': '
    Nagaland', 'climate': 'moderate', 'best_time': 'Oct - Apr', '
    description': 'Cultural hub and festival destination'},
116         {'name': 'Puri', 'image_file': 'puri.jpg', 'state': 'Odisha',
     'climate': 'hot', 'best_time': 'Oct - Mar', 'description': '
    Sacred temple and beach town'},
```

```
117         {'name': 'Golden Temple', 'image_file': 'golden_temple.jpg',
    'state': 'Punjab', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'Holy pilgrimage site and spiritual center'},
118         {'name': 'Jaipur', 'image_file': 'jaipur.jpg', 'state': '
    Rajasthan', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'City palace and historic forts'},
119         {'name': 'Gangtok', 'image_file': 'gangtok.jpg', 'state': '
    Sikkim', 'climate': 'moderate', 'best_time': 'Mar - Jun', '
    description': 'Mountain city with Buddhist monasteries'},
120         {'name': 'Mahabalipuram', 'image_file': 'mahabalipuram.jpg',
    'state': 'Tamil Nadu', 'climate': 'hot', 'best_time': 'Nov - Feb',
     'description': 'Heritage temples and beach town'},
121         {'name': 'Hyderabad', 'image_file': 'hyderabad.jpg', 'state':
     'Telangana', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'Historic monuments and tech city'},
122         {'name': 'Ujjayanta Palace', 'image_file': 'ujjayanta_palace.
    jpg', 'state': 'Tripura', 'climate': 'hot', 'best_time': 'Oct -
    Mar', 'description': 'Historic palace and museum'},
123         {'name': 'Agra', 'image_file': 'agra.jpg', 'state': 'Uttar
    Pradesh', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description
    ': 'Taj Mahal and iconic monument'},
124         {'name': 'Rishikesh', 'image_file': 'rishikesh.jpg', 'state':
     'Uttarakhand', 'climate': 'moderate', 'best_time': 'Sep - Nov', '
    description': 'Yoga capital and adventure hub'},
125         {'name': 'Darjeeling', 'image_file': 'darjeeling.jpg', 'state
    ': 'West Bengal', 'climate': 'cold', 'best_time': 'Mar - Jun', '
    description': 'Tea gardens and mountain railway'},
126         {'name': 'Port Blair', 'image_file': 'port_blair.jpg', 'state
    ': 'Andaman and Nicobar Islands', 'climate': 'hot', 'best_time': '
    Oct - May', 'description': 'Island paradise and beaches'},
127         {'name': 'Rock Garden', 'image_file': 'rock_garden.jpg', '
    state': 'Chandigarh', 'climate': 'moderate', 'best_time': 'Sep -
    Mar', 'description': 'Artistic rock garden creation'},
128         {'name': 'Daman Beach', 'image_file': 'daman_beach.jpg', '
    state': 'Dadra and Nagar Haveli and Daman and Diu', 'climate': '
    hot', 'best_time': 'Oct - Mar', 'description': 'Beach resort and
    watersports'},
129         {'name': 'Srinagar', 'image_file': 'srinagar.jpg', 'state': '
    Jammu and Kashmir', 'climate': 'cold', 'best_time': 'Apr - Oct', '
    description': 'Houseboats and scenic lakes'},
130         {'name': 'Pangong Lake', 'image_file': 'pangong_lake.jpg', '
    state': 'Ladakh', 'climate': 'cold', 'best_time': 'Jun - Sep', '
    description': 'High altitude lake and trekking'},
131         {'name': 'Minicoy Island', 'image_file': 'minicoy_island.jpg'
    , 'state': 'Lakshadweep', 'climate': 'hot', 'best_time': 'Oct -
    Mar', 'description': 'Tropical island paradise'},
132         {'name': 'Pondicherry', 'image_file': 'pondicherry.jpg', '
```

```
    state': 'Puducherry', 'climate': 'hot', 'best_time': 'Oct - Mar',
    'description': 'French colonial charm and beaches'},
133         {'name': 'India Gate', 'image_file': 'india_gate.jpg', 'state
    ': 'Delhi', 'climate': 'hot', 'best_time': 'Oct - Mar', '
    description': 'Iconic monument and city landmark'},
134     ]
135     # Add a cache-busting version (file mtime) for each image so
    browser picks up updates
136     for d in destinations_data:
137         filename = d.get('image_file')
138         if filename:
139             local_path = os.path.join(settings.BASE_DIR, 'static', '
    images', filename)
140             try:
141                 d['image_version'] = str(int(os.path.getmtime(
    local_path)))
142             except Exception:
143                 d['image_version'] = '0'
144         else:
145             d['image_version'] = '0'
146
147
148     return render(request, "destinations.html", {'destinations':
    destinations_data})
149
150
151 @login_required(login_url='login')
152 def plan_trip_page(request):
153     """Display trip planning form"""
154     return render(request, "plan_trip.html")
155
156
157 @login_required(login_url='login')
158 def my_trips_page(request):
159     """Display a list of saved trips"""
160     trips = Trip.objects.filter(is_deleted=False).order_by('-
    created_at')
161     deleted_trips = Trip.objects.filter(is_deleted=True).order_by('-
    deleted_at')
162     return render(request, "my_trips.html", {"trips": trips, "
    deleted_trips": deleted_trips})
163
164
165 @login_required(login_url='login')
166 def delete_trip(request, trip_id):
167     """Soft delete a trip"""
168     if request.method == 'POST':
```

```
        try:
            trip = get_object_or_404(Trip, pk=trip_id, is_deleted=
    False)
            trip.delete()  # This will now do soft delete
            messages.success(request, f'Trip to {trip.destination}
    has been moved to trash.')
        except Trip.DoesNotExist:
            messages.error(request, 'Trip not found.')
    return redirect('my-trips')


@login_required(login_url='login')
def recycle_trip(request, trip_id):
    """Restore a soft-deleted trip"""
    if request.method == 'POST':
        try:
            trip = get_object_or_404(Trip, pk=trip_id, is_deleted=
    True)
            trip.recycle()
            messages.success(request, f'Trip to {trip.destination}
    has been restored.')
        except Trip.DoesNotExist:
            messages.error(request, 'Trip not found.')
    return redirect('my-trips')


@login_required(login_url='login')
def trip_detail_page(request, trip_id):
    """Display a saved trip's details"""
    trip = get_object_or_404(Trip, pk=trip_id)
    trip_plan = trip.plan_data or {}
    # If we have older trips in the DB without full plan_data,
    backfill key pieces
    if not trip_plan.get('destination'):
        trip_plan['destination'] = trip.destination
    if not trip_plan.get('budget'):
        trip_plan['budget'] = trip.budget
    if not trip_plan.get('days'):
        trip_plan['days'] = trip.days
    if not trip_plan.get('climate'):
        trip_plan['climate'] = trip.climate
    if not trip_plan.get('travel_type'):
        trip_plan['travel_type'] = trip.travel_type


    if not trip_plan.get('itinerary'):
        try:
```

```
            # Try to parse as JSON array first
            parsed_itinerary = json.loads(trip.itinerary)
            if isinstance(parsed_itinerary, list):
                trip_plan['itinerary'] = parsed_itinerary
            else:
                # If it's a string, convert to single-item list
                trip_plan['itinerary'] = [trip.itinerary]
        except (json.JSONDecodeError, TypeError):
            # If JSON parsing fails, treat as plain string and convert to list
            trip_plan['itinerary'] = [trip.itinerary] if trip.itinerary else []


    # Add image URL if exists
    if trip.image:
        trip_plan['image_url'] = trip.image.url


    trip_plan["trip_id"] = trip.id


    # Keep the plan in session to support the results template
    request.session["trip_plan"] = trip_plan
    return render(request, "trip_results.html", {"trip_plan": trip_plan})


@login_required(login_url='login')
def trip_results_page(request):
    """Display trip results"""


    # Allow viewing a saved trip by id
    trip_id = request.GET.get('trip_id')
    if trip_id:
        try:
            trip = Trip.objects.get(pk=int(trip_id))
            trip_data = trip.plan_data or {}
            # Backfill missing fields for trips created before plan_data was added
            if not trip_data.get('destination'):
                trip_data['destination'] = trip.destination
            if not trip_data.get('budget'):
                trip_data['budget'] = trip.budget
            if not trip_data.get('days'):
                trip_data['days'] = trip.days
```

```
            if not trip_data.get('climate'):
                trip_data['climate'] = trip.climate
            if not trip_data.get('travel_type'):
                trip_data['travel_type'] = trip.travel_type


            if not trip_data.get('itinerary'):
                try:
                    # Try to parse as JSON array first
                    parsed_itinerary = json.loads(trip.itinerary)
                    if isinstance(parsed_itinerary, list):
                        trip_data['itinerary'] = parsed_itinerary
                    else:
                        # If it's a string, convert to single-item list
                        trip_data['itinerary'] = [trip.itinerary]
                except (json.JSONDecodeError, TypeError):
                    # If JSON parsing fails, treat as plain string and convert to list
                    trip_data['itinerary'] = [trip.itinerary] if trip.itinerary else []


            # Add image URL if exists
            if trip.image:
                trip_data['image_url'] = trip.image.url


            trip_data["trip_id"] = trip.id
            request.session["trip_plan"] = trip_data
        except (Trip.DoesNotExist, ValueError):
            pass


    # Check if trip data is passed via URL parameter
    trip_data_param = request.GET.get('data')
    if trip_data_param:
        try:
            trip_data = json.loads(unquote(trip_data_param))
            request.session["trip_plan"] = trip_data
        except (json.JSONDecodeError, TypeError):
            pass


    # Get trip data from session
    trip_data = request.session.get("trip_plan")
    if not trip_data:
```

```
        # If no trip data, redirect to plan trip page
        return redirect('plan-trip')


    return render(request, "trip_results.html", {'trip_plan': trip_data})


class TripRecommendationView(APIView):
    @method_decorator(csrf_exempt)
    def dispatch(self, *args, **kwargs):
        return super().dispatch(*args, **kwargs)


    def post(self, request):
        serializer = TripRequestSerializer(data=request.data)


        if serializer.is_valid():
            data = serializer.validated_data


            # If destination is provided, use it; otherwise predict one
            if 'destination' in request.data and request.data['destination']:
                destination = request.data['destination']
            else:
                destination = predict_destination(
                    budget=data["budget"],
                    days=data["days"],
                    climate=data["climate"],
                    travel_type=data["travel_type"]
                )


            # Parse interests if it's a JSON string
            interests = request.data.get('interests', [])
            if isinstance(interests, str):
                try:
                    interests = json.loads(interests)
                except json.JSONDecodeError:
                    interests = []


            # Convert travelers to int
            try:
```

```
                travelers = int(request.data.get('travelers', 1))
            except (ValueError, TypeError):
                travelers = 1


            try:
                # Generate comprehensive trip plan
                trip_plan = generate_trip_plan(
                    destination=destination,
                    days=data["days"],
                    budget=data["budget"],
                    climate=data["climate"],
                    travel_type=data["travel_type"],
                    travelers=travelers,
                    accommodation=request.data.get('accommodation', 'standard'),
                    interests=interests,
                    special_requests=request.data.get('special_requests', '')
                )


                # Save to database (including full plan data for later review)
                trip = Trip.objects.create(
                    destination=trip_plan['destination'],
                    budget=trip_plan['budget'],
                    days=trip_plan['days'],
                    climate=trip_plan['climate'],
                    travel_type=trip_plan['travel_type'],
                    itinerary=json.dumps(trip_plan.get('itinerary', [])),
                    plan_data=trip_plan,
                    image=data.get('image'),  # Save uploaded image if provided
                )


                # Return full trip plan data
                return Response({
                    **trip_plan,
                    "trip_id": trip.id,
                }, status=status.HTTP_200_OK)
            except Exception as e:
                return Response({'error': str(e)}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

```

```
383
384         return Response(serializer.errors, status=status.
    HTTP_400_BAD_REQUEST)
```

Listing A.7: views.py

A.1.8 `Manage.py`

```
1  #!/usr/bin/env python
2  """Django's command-line utility for administrative tasks."""
3  import os
4  import sys
5
6
7  def main():
8      """Run administrative tasks."""
9      os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings'
       )
10     try:
11         from django.core.management import execute_from_command_line
12     except ImportError as exc:
13         raise ImportError(
14             "Couldn't import Django. Are you sure it's installed and
       "
15             "available on your PYTHONPATH environment variable? Did
       you "
16             "forget to activate a virtual environment?"
17         ) from exc
18     execute_from_command_line(sys.argv)
19
20
21 if __name__ == '__main__':
22     main()
```

Listing A.8: Manage.py

A.2 Frontend Templates

A.2.1 `base.html`

```
1  {% load static %}
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
```

```
    <meta charset="UTF-8">
    <title>AI Trip Planner</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">


    <!-- Bootstrap 5 CDN -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
    <link rel="stylesheet" href="{% static 'css/style.css' %}">

    <!-- CSRF Token for AJAX requests -->
    <meta name="csrf-token" content="{{ csrf_token }}">
</head>
<body>
    <!-- Hidden form for CSRF token -->
    <form id="csrf-form" style="display: none;">
        {% csrf_token %}
    </form>


<!-- ♦ Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow">
    <div class="container">
        <a class="navbar-brand fw-bold" href="{% url 'home' %}">✈ AI Trip Planner</a>


        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
                data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
            <span class="navbar-toggler-icon"></span>
        </button>


        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ms-auto">
                {% if user.is_authenticated %}
                    <li class="nav-item">
                        <a class="nav-link" href="{% url 'home' %}">Home</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{% url 'destinations' %}">Destinations</a>
                    </li>
```

```
                    <li class="nav-item">
                        <a class="nav-link" href="{% url 'plan-trip' %}">Plan Trip</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{% url 'my-trips' %}">My Trips</a>
                    </li>

                    <li class="nav-item dropdown">
                        <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-toggle="dropdown" aria-expanded="false">
                            👤 {{ user.username }}
                        </a>
                        <ul class="dropdown-menu dropdown-menu-end" aria-labelledby="navbarDropdown">
                            <li><a class="dropdown-item" href="{% url 'my-trips' %}">My Trips</a></li>
                            <li><hr class="dropdown-divider"></li>
                            <li><a class="dropdown-item" href="{% url 'logout' %}">Logout</a></li>
                        </ul>
                    </li>
                {% else %}
                    <li class="nav-item">
                        <a class="nav-link" href="{% url 'login' %}">Login</a>
                    </li>
                    <li class="nav-item">
                        <a class="btn btn-primary ms-2" href="{% url 'register' %}">Sign Up</a>
                    </li>
                {% endif %}
            </ul>
        </div>
    </div>
</nav>


<!-- Messages -->
{% if messages %}
<div class="container mt-3">
    {% for message in messages %}
    <div class="alert alert-{{ message.tags }} alert-dismissible fade show" role="alert">
        {{ message }}
```

```
        <button type="button" class="btn-close" data-bs-dismiss="
    alert" aria-label="Close"></button>
    </div>
    {% endfor %}
</div>
{% endif %}


{% block content %}{% endblock %}


<!-- ♦ Footer -->
<footer class="bg-dark text-white text-center p-4 mt-5">
    <p class="mb-0">© 2026 AI Trip Planner</p>
</footer>


<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/
    bootstrap.bundle.min.js"></script>
</body>
</html>
```

Listing A.9: base.html

A.2.2 Destinations.html

```
{% extends 'base.html' %}
{% load static %}


{% block content %}


<!-- 🌍 Destinations Header -->
<div class="bg-light py-5">
    <div class="container">
        <h1 class="display-5 fw-bold mb-3">Explore Destinations</h1>
        <p class="lead text-muted">Discover amazing places across
    India</p>
    </div>
</div>


<!-- 🔍 Filter Section -->
<section class="container py-4">
    <div class="row mb-4">
```

```
        <div class="col-md-4">
            <input type="text" id="searchInput" class="form-control form-control-lg" placeholder="Search destinations...">
        </div>
        <div class="col-md-4">
            <select id="climateFilter" class="form-select form-select-lg">
                <option value="">All Climates</option>
                <option value="hot">Hot</option>
                <option value="cold">Cold</option>
                <option value="moderate">Moderate</option>
            </select>
        </div>
        <div class="col-md-4">
            <button class="btn btn-outline-primary w-100" onclick="resetFilters()">Reset Filters</button>
        </div>
    </div>
</section>


<!-- 🖼️ Destinations Grid -->
<section class="container pb-5">
    <div class="row g-4" id="destinationsContainer">
        {% for destination in destinations %}
        <div class="col-md-6 col-lg-4 destination-card" data-name="{{ destination.name|lower }}" data-climate="{{ destination.climate }}">
            <div class="card h-100 shadow-sm border-0 transition-card">
                <img src="{% static 'images/' %}{{ destination.image_file }}?v={{ destination.image_version }}" class="card-img-top" alt="{{ destination.name }}" style="height: 250px; object-fit: cover;" onerror="this.onerror=null; this.src='https://via.placeholder.com/400x250?text=No+Image';">
                <div class="card-body">
                    <h5 class="card-title fw-bold">{{ destination.name }}</h5>
                    <p class="card-text text-muted small">{{ destination.state }}</p>
                    <p class="card-text text-muted">{{ destination.description }}</p>

                    <div class="mb-3">
                        <span class="badge bg-info">{{ destination.climate|upper }}</span>
                        <span class="badge bg-warning text-dark">{{
```

```
    destination.best_time }}</span>
53                      </div>
54                  </div>
55                  <div class="card-footer bg-white border-top-0">
56                      <a href="{% url 'plan-trip' %}?destination={{
    destination.name }}" class="btn btn-primary w-100">Plan Trip Here<
    /a>
57                  </div>
58              </div>
59          </div>
60          {% endfor %}
61      </div>
62
63
64      <!-- No Results Message -->
65      <div id="noResults" class="alert alert-info text-center py-5"
    style="display: none;">
66          <h5>No destinations found</h5>
67          <p>Try adjusting your filters</p>
68      </div>
69  </section>
70
71
72  <!-- 📱 Features Section -->
73  <section class="bg-light py-5">
74      <div class="container">
75          <h2 class="text-center mb-5">Why Choose Our Destinations?</h2
    >
76          <div class="row g-4">
77              <div class="col-md-4 text-center">
78                  <div class="mb-3">
79                      <i class="bi bi-star-fill" style="font-size: 2rem
    ; color: #007bff;"></i>
80                  </div>
81                  <h5>Curated Selections</h5>
82                  <p>Handpicked destinations for unforgettable
    experiences</p>
83              </div>
84              <div class="col-md-4 text-center">
85                  <div class="mb-3">
86                      <i class="bi bi-person-check" style="font-size: 2
    rem; color: #28a745;"></i>
87                  </div>
88                  <h5>Expert Recommendations</h5>
89                  <p>AI-powered suggestions based on your preferences</
    p>
90              </div>
```

```
            <div class="col-md-4 text-center">
                <div class="mb-3">
                    <i class="bi bi-calendar-check" style="font-size: 2rem; color: #ffc107;"></i>
                </div>
                <h5>Best Time to Visit</h5>
                <p>Know the perfect season for each destination</p>
            </div>
        </div>
    </div>
</section>


<!-- 🎯 CTA Section -->
<section class="bg-primary text-white py-5">
    <div class="container text-center">
        <h2 class="mb-3">Ready to Plan Your Trip?</h2>
        <p class="lead mb-4">Let our AI help you create the perfect itinerary</p>
        <a href="{% url 'plan-trip' %}" class="btn btn-light btn-lg">Start Planning Now</a>
    </div>
</section>


<style>
    .transition-card {
        transition: transform 0.3s ease, box-shadow 0.3s ease;
    }


    .transition-card:hover {
        transform: translateY(-10px);
        box-shadow: 0 15px 30px rgba(0, 0, 0, 0.2) !important;
    }


    .destination-card {
        animation: fadeIn 0.5s ease-in;
    }


    @keyframes fadeIn {
        from {
            opacity: 0;
            transform: translateY(10px);
        }
```

```
        to {
            opacity: 1;
            transform: translateY(0);
        }
    }
</style>


<script>
    const searchInput = document.getElementById('searchInput');
    const climateFilter = document.getElementById('climateFilter');
    const destinationCards = document.querySelectorAll('.destination-card');
    const noResults = document.getElementById('noResults');


    function filterDestinations() {
        const searchTerm = searchInput.value.toLowerCase();
        const selectedClimate = climateFilter.value;
        let visibleCount = 0;


        destinationCards.forEach(card => {
            const name = card.getAttribute('data-name');
            const climate = card.getAttribute('data-climate');

            const matchesSearch = name.includes(searchTerm);
            const matchesClimate = selectedClimate === '' || climate === selectedClimate;


            if (matchesSearch && matchesClimate) {
                card.style.display = 'block';
                visibleCount++;
            } else {
                card.style.display = 'none';
            }
        });


        noResults.style.display = visibleCount === 0 ? 'block' : 'none';
    }


    function resetFilters() {
        searchInput.value = '';
```

```
        climateFilter.value = '';
        filterDestinations();
    }


    searchInput.addEventListener('keyup', filterDestinations);
    climateFilter.addEventListener('change', filterDestinations);
</script>


{% endblock %}
```

Listing A.10: Destinations.html

A.2.3 Home.html

```
{% extends 'base.html' %}
{% load static %}


{% block content %}


<!-- 🖼️ Hero Section -->
<section class="hero-section text-white text-center d-flex align-items-center">
    <div class="container">
        <h1 class="display-4 fw-bold">Plan Your Perfect Trip with AI</h1>
        <p class="lead">Smart itineraries. Smart budgets. Smart travel.</p>
        <a href="#planner" class="btn btn-primary btn-lg mt-3">Start Planning</a>
    </div>
</section>


<!-- 🎞️ Carousel -->
<div class="container mt-5">
    <h2 class="text-center mb-4">Popular Destinations</h2>


    <div id="destinationCarousel" class="carousel slide" data-bs-ride="carousel">
        <div class="carousel-inner rounded shadow">

```

```
26
27              <div class="carousel-item active">
28                  <img src="{% static 'css/images/Goa.jpeg' %}" class="
    d-block w-100" alt="Goa">
29                  <div class="carousel-caption">
30                      <h3>Goa</h3>
31                  </div>
32              </div>
33
34
35              <div class="carousel-item">
36                  <img src="{% static 'css/images/Manali.jpeg' %}"
    class="d-block w-100" alt="Manali">
37                  <div class="carousel-caption">
38                      <h3>Manali</h3>
39                  </div>
40              </div>
41
42
43              <div class="carousel-item">
44                  <img src="{% static 'css/images/Jaipur.jpg' %}" class
    ="d-block w-100" alt="Jaipur">
45                  <div class="carousel-caption">
46                      <h3>Jaipur</h3>
47                  </div>
48              </div>
49
50
51          </div>
52
53
54          <button class="carousel-control-prev" type="button" data-bs-
    target="#destinationCarousel" data-bs-slide="prev">
55              <span class="carousel-control-prev-icon"></span>
56          </button>
57
58
59          <button class="carousel-control-next" type="button" data-bs-
    target="#destinationCarousel" data-bs-slide="next">
60              <span class="carousel-control-next-icon"></span>
61          </button>
62      </div>
63  </div>
64
65
66  <!-- 📄 Planner Form -->
67  <section id="planner" class="container mt-5">
```

```
 <div class="card shadow-lg p-4">
     <h3 class="mb-4 text-center">Generate Your Trip</h3>


     <form id="tripForm">
         <div class="row">
             <div class="col-md-6 mb-3">
                 <label class="form-label">Number of Days</label>
                 <input type="number" class="form-control" id="
days" name="days" min="1" max="30" required>
             </div>


             <div class="col-md-6 mb-3">
                 <label class="form-label">Budget (Rs.)</label>
                 <input type="number" class="form-control" id="
budget" name="budget" min="1000" required>
             </div>
         </div>


         <div class="row">
             <div class="col-md-6 mb-3">
                 <label class="form-label">Climate Preference</
label>
                 <select class="form-select" id="climate" name="
climate" required>
                     <option value="">Select Climate</option>
                     <option value="hot">Hot</option>
                     <option value="cold">Cold</option>
                     <option value="moderate">Moderate</option>
                 </select>
             </div>


             <div class="col-md-6 mb-3">
                 <label class="form-label">Travel Type</label>
                 <select class="form-select" id="travel_type" name
="travel_type" required>
                     <option value="">Select Travel Type</option>
                     <option value="adventure">Adventure</option>
                     <option value="relaxation">Relaxation</option
>
                 </select>
             </div>
         </div>

```

```
109
110         <button type="submit" class="btn btn-primary w-100">Generate Recommendation</button>
111     </form>
112
113
114     <div id="result" class="mt-4" style="display:none;">
115         <div class="alert alert-success">
116             <h5>✈ Recommended Destination</h5>
117             <h2 id="destination" class="text-primary"></h2>
118             <p><strong>Budget:</strong> Rs.<span id="resultBudget"></span></p>
119             <p><strong>Days:</strong> <span id="resultDays"></span></p>
120             <p><strong>Climate:</strong> <span id="resultClimate"></span></p>
121             <p><strong>Travel Type:</strong> <span id="resultTravelType"></span></p>
122         </div>
123     </div>
124
125
126     <script>
127         document.getElementById('tripForm').addEventListener('submit', async function(e) {
128             e.preventDefault();
129
130             const formData = {
131                 days: parseInt(document.getElementById('days').value),
132                 budget: parseFloat(document.getElementById('budget').value),
133                 climate: document.getElementById('climate').value,
134                 travel_type: document.getElementById('travel_type').value
135             };
136
137
138             try {
139                 const endpoint = '{% url "recommend-trip" %}';
140                 const response = await fetch(endpoint, {
141                     method: 'POST',
142                     headers: {
143                         'Content-Type': 'application/json',
144                         'X-CSRFToken': getCookie('csrftoken')
145                     },
```

```
                        body: JSON.stringify(formData)
                    });


                    const result = await response.json();


                    if (response.ok) {
                        const tripData = encodeURIComponent(JSON.
    stringify(result));
                        window.location.href = `{% url "trip-results"
     %}?data=${tripData}`;
                    } else {
                        alert('Error: ' + JSON.stringify(result));
                    }
                } catch (error) {
                    alert('Error: ' + error.message);
                }
            });


            function getCookie(name) {
                let cookieValue = null;
                if (document.cookie && document.cookie !== '') {
                    const cookies = document.cookie.split(';');
                    for (let i = 0; i < cookies.length; i++) {
                        const cookie = cookies[i].trim();
                        if (cookie.substring(0, name.length + 1) ===
    (name + '=')) {
                            cookieValue = decodeURIComponent(cookie.
    substring(name.length + 1));
                            break;
                        }
                    }
                }
                return cookieValue;
            }
        </script>
    </div>
</section>


{% endblock %}
```

Listing A.11: Home.html

A.2.4 Mytrips.html

```
{% extends 'base.html' %}
{% load humanize %}


{% block content %}
<div class="container py-5">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <div>
      <h1 class="h3">My Saved Trips</h1>
      <p class="text-muted mb-0">Review and revisit past trip plans.</p>
    </div>
    <a href="{% url 'plan-trip' %}" class="btn btn-primary">Plan a New Trip</a>
  </div>


  {% if trips %}
  <div class="table-responsive">
    <table class="table table-hover align-middle">
      <thead class="table-light">
        <tr>
          <th>Photo</th>
          <th>Destination</th>
          <th>Created</th>
          <th>Budget</th>
          <th>Days</th>
          <th>Type</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {% for trip in trips %}
        <tr>
          <td>
            {% if trip.image %}
              <img src="{{ trip.image.url }}" alt="Trip Photo" class="rounded" style="width: 60px; height: 40px; object-fit: cover;">
            {% else %}
              <div class="bg-light rounded d-flex align-items-center justify-content-center" style="width: 60px; height: 40px;">
                <span class="text-muted small">📷</span>
              </div>
            {% endif %}
```

```
          </td>
          <td>{{ trip.destination }}</td>
          <td>{{ trip.created_at|date:"M d, Y H:i" }}</td>
          <td>Rs.{{ trip.budget|intcomma }}</td>
          <td>{{ trip.days }}</td>
          <td>{{ trip.travel_type|title }}</td>
          <td class="text-end">
            <a href="{% url 'trip-detail' trip.id %}" class="btn btn-sm btn-outline-primary me-1">View</a>
            <form method="post" action="{% url 'delete-trip' trip.id %}" class="d-inline" onsubmit="return confirm('Are you sure you want to delete this trip?')">
              {% csrf_token %}
              <button type="submit" class="btn btn-sm btn-outline-danger">Delete</button>
            </form>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
  {% else %}
  <div class="card border-0 shadow-sm">
    <div class="card-body text-center">
      <h5 class="card-title">No saved trips yet</h5>
      <p class="card-text text-muted">Plan a trip to see it listed here.</p>
      <a href="{% url 'plan-trip' %}" class="btn btn-primary">Start Planning</a>
    </div>
  </div>
  {% endif %}


  {% if deleted_trips %}
  <div class="mt-5">
    <h4 class="h5 text-muted mb-3">🗑 Recently Deleted Trips</h4>
    <div class="table-responsive">
      <table class="table table-hover align-middle opacity-75">
        <thead class="table-light">
          <tr>
            <th>Photo</th>
            <th>Destination</th>
            <th>Deleted</th>
            <th>Budget</th>
            <th>Days</th>
```

```
            <th>Type</th>
            <th></th>
          </tr>
        </thead>
        <tbody>
          {% for trip in deleted_trips %}
          <tr>
            <td>
              {% if trip.image %}
                <img src="{{ trip.image.url }}" alt="Trip Photo"
    class="rounded opacity-50" style="width: 60px; height: 40px;
    object-fit: cover;">
              {% else %}
                <div class="bg-light rounded d-flex align-items-
    center justify-content-center opacity-50" style="width: 60px;
    height: 40px;">
                  <span class="text-muted small">📷</span>
                </div>
              {% endif %}
            </td>
            <td class="text-muted">{{ trip.destination }}</td>
            <td class="text-muted">{{ trip.deleted_at|date:"M d, Y H:
    i" }}</td>
            <td class="text-muted">Rs.{{ trip.budget|intcomma }}</td>
            <td class="text-muted">{{ trip.days }}</td>
            <td class="text-muted">{{ trip.travel_type|title }}</td>
            <td class="text-end">
              <form method="post" action="{% url 'recycle-trip' trip.
    id %}" class="d-inline">
                {% csrf_token %}
                <button type="submit" class="btn btn-sm btn-outline-
    success">Recycle</button>
              </form>
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
  </div>
  {% endif %}
</div>
{% endblock %}
```

Listing A.12: Mytrips.html

Appendix B

Calibration and Setup Guide

This appendix provides a step-by-step guide for setting up, calibrating, and running the AI-Based Trip Planner Web Application on a local development environment.

B.1 Prerequisites

Before you start, ensure you have the following installed on your system:

- **Python**: Version 3.8 or higher
- **PostgreSQL**: Running locally on port 5432
- **Git**: Optional, for version control

B.2 Database Setup

This project uses PostgreSQL. You need to create a database and user matching the configuration in `config/settings.py`.

1. Open your terminal or command prompt.
2. Log into the PostgreSQL interactive terminal:

```
psql -U postgres

```

3. Create the database:

```
CREATE DATABASE ai_project_db;

```

4. Verify that the user `postgres` has the password `postgres`. If you need to alter the password:

```
ALTER USER postgres WITH PASSWORD 'postgres';
```

5. Exit the PostgreSQL terminal:

```
\q
```

B.3 Environment Setup

It's recommended to run the project inside an isolated virtual environment. The project is already configured with an environment directory named `myenv`.

1. Navigate to the project root directory:

```
cd /path/to/AI_Project
```

2. Activate the virtual environment:

   - On macOS/Linux:

```
source myenv/bin/activate
```

   - On Windows:

```
myenv\Scripts\activate
```

B.4 Install Dependencies

A `requirements.txt` file is present in the project directory with all the required dependencies. With the virtual environment activated, install the required packages using `pip`:

```
pip install -r requirements.txt
```

B.5 Apply Migrations

Set up the database schema by applying the Django migrations:

```
python manage.py makemigrations
python manage.py migrate
```

B.6 Create a Superuser (Optional but Recommended)

To access the Django admin panel, you'll need an administrative user:

```
python manage.py createsuperuser
```

Follow the prompts to set your username, email, and password.

B.7 Run the Development Server

Finally, start the local server:

```
python manage.py runserver
```

The application should now be accessible at `http://127.0.0.1:8000/`.

You can log into the admin interface at `http://127.0.0.1:8000/admin/` using the superuser credentials you created.

Note

The system is set up to look for static files in the `static` and `static-files` directories. If you make any CSS or JS changes, they will be loaded dynamically in development mode.